



Awarded by



LSTM vs. Random Forest for Solar Power Forecasting Across Multiple Plants

A thesis submitted by

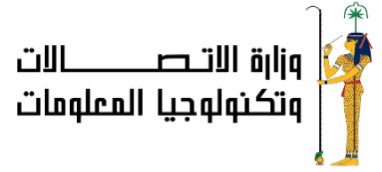
Mostafa Mahmoud Mohamed	11005
Ahmed Hassan Omran	11106
Mohamed Abdelhalim Ragab	11111
Mostafa Khalifa Mohamed	11006
Youssef Mahmoud Ali	11003

**A dissertation submitted in partial fulfillment of the requirements for the degree of
Digilians 9 Months Diploma in Applied AI and Data Analytics**

Under supervision of
Maj. Gen. (R.) Prof. Dr. Gouda Ismail Salama
Military Technical College

Eng. Omar Khalid
Military Technical College

(August 2026)



Awarded by



Approval Sheet

Solar Power Forecasting at 15-Minute and 24-Hour Horizons: A Comparative Analysis of LSTM and Random Forest Approaches

A thesis submitted by

Mostafa Mahmoud Mohamed	11005
Ahmed Hassan Omran	11106
Mohamed Abdelhalim Ragab	11111
Mostafa Khalifa Mohamed	11006
Youssef Mahmoud Ali	11003

A dissertation submitted in partial fulfillment of the requirements for the
degree of

Digilians 9 Months Diploma

Applied AI and Data Analytics Track

Approved by the examination committee

Name

Signature

Prof.

Prof.

Prof.

Abstract

The increasing integration of solar photovoltaic (PV) systems into modern power grids has created a critical demand for accurate and reliable power generation forecasting. Uncertainty in solar output — driven by weather variability, cloud cover, and plant-specific characteristics — poses significant challenges to grid operators responsible for balancing supply and demand in real time.

This project presents a deep learning forecasting system for solar PV power output, developed and evaluated on the BR-PVGen dataset comprising 51 photovoltaic plants across Brazil, recorded at 15-minute intervals. The system addresses two forecast horizons simultaneously: 15-minute-ahead prediction, enabling direct comparison with published benchmarks, and 24-hour-ahead prediction, which represents a more operationally demanding and practically relevant task.

The proposed architecture is a dual-input Long Short-Term Memory (LSTM) network. The first input branch processes a 48-hour historical window of weather and power measurements — including plane-of-array irradiance, ambient and panel temperature, wind speed and direction, and cyclic temporal encodings — as a sequential time-series signal. The second input branch encodes plant-specific metadata, including nominal capacity, panel efficiency, and structural type, through a dedicated dense embedding layer. Both branches are merged before a shared prediction head.

To address the heterogeneity of the 51 plants, a Transfer Learning strategy is applied following global pre-training. The LSTM layers — which learn general solar temporal dynamics shared across all plants — are frozen, while the dense prediction head is individually fine-tuned on each plant's own data. This allows data-poor plants to benefit from the knowledge extracted across the full dataset while capturing their unique operational characteristics.

The 15-minute model achieves a SMAPE of 32.1%, outperforming all six baseline models reported in the reference paper, including the best-performing MLP with Transfer Learning (43.9%). The 24-hour model achieves a pooled R^2 of 0.789 across all 51 plants, demonstrating strong generalization despite the inherent difficulty of the longer horizon. The trained system is deployed as a production-ready FastAPI backend, capable of serving real-time predictions for both known plants and previously unseen installations worldwide.

Acknowledgments

We would like to express our sincere gratitude to our supervisor, Maj. Gen. (R.) Prof. Dr. Gouda Ismail Salama, for their continuous guidance, constructive feedback, and unwavering support throughout the duration of this project. Their expertise and encouragement were invaluable in shaping both the technical direction and the academic quality of this work.

We are grateful to the creators and maintainers of the BR-PVGen dataset for making this comprehensive collection of Brazilian photovoltaic plant data publicly available to the research community. Without this dataset, the scope and depth of this work would not have been achievable.

We also acknowledge the open-source communities behind TensorFlow, Keras, FastAPI, pandas, and scikit-learn, whose tools formed the foundation of our implementation.

Finally, we would like to thank our families and colleagues for their patience, moral support, and encouragement throughout the long hours of development, experimentation, and writing that this project demanded.

Table of Contents

Abstract	I
Acknowledgments	II
List of Figures	VII
List of Tables	VIII
Chapter 1: Introduction	1
1.1 Overview	1
1.2 Motivation and Challenges	2
1.3 Problem Statement	3
1.4 Proposed Solution	4
1.5 Thesis Organization	4
Chapter 2: Background and Literature Review	6
2.1 Introduction	6
2.2 Solar PV Power Forecasting Overview	6
2.2.1 Forecasting Horizons (Intra-day vs. Day-ahead)	7
2.2.2 Challenges in PV Power Forecasting	7
2.2.3 Transfer Learning in Time-Series Forecasting	9
2.3 Machine Learning Overview	10
2.3.1 Fundamental Terms of Machine Learning	10
2.3.2 Regression and Time-Series Algorithms	11
2.3.3 Random Forest Algorithm	11
2.3.4 Gradient Boosting / XGBoost Algorithm	12
2.3.5 ARIMA Algorithm	12
2.3.6 Recurrent Neural Networks and LSTM	13
2.4 Literature Review	14
CHAPTER 3:	17
PROPOSED DUAL-HORIZON SOLAR POWER FORECASTING APPROACH	17
3.1 Introduction	17
3.2 Proposed Forecasting Scenarios	17
3.2.1 15-Minute-Ahead Forecasting	18
3.2.2 24-Hour-Ahead Forecasting	18
3.2.3 24-Hour Multi-Step Forecasting (96 Steps)	19
3.3 Proposed System Workflow	19
3.4 BR-PVGen Dataset	20
3.4.1 Dataset Description	20
3.4.2 Solar Power Generation Data	20
3.4.3 Weather and Environmental Data	21
3.4.4 Data Structure and Sampling Interval	21
3.5 Data Preparation	22
3.5.1 Data Loading and Integration	22
3.5.2 Date-Time Processing and Time Alignment	22

3.5.3 Handling Missing and Invalid Values	23
3.5.4 Data Splitting	23
3.5.5 Feature Scaling and Normalization	23
3.6 Feature Engineering	24
3.6.1 Temporal Features	24
3.6.2 Cyclic Time Encoding	24
3.6.3 Solar Irradiance Features	25
3.6.4 Historical Power Features	25
3.6.5 Target Variable Construction	25
3.7 Input and Output Configuration	26
3.7.1 Input Sequence Construction	26
3.7.2 15-Minute-Ahead Target	27
3.7.3 24-Hour-Ahead Target	27
3.8 Proposed Forecasting Models	27
3.8.1 LSTM-Based Forecasting Model	27
3.8.2 Random Forest Regressor	28
3.8.3 Comparison of LSTM and Random Forest	28
3.9 LSTM Model Architecture	29
3.9.1 Time-Series Input	29
3.9.2 LSTM Layers	29
3.9.3 Metadata / Plant Information	30
3.9.4 Dense Layers and Output Layer	30
3.10 Random Forest Model Design	30
3.10.1 Input Features	31
3.10.2 Random Forest Regression	31
3.10.3 Prediction Strategy for Different Horizons	32
3.11 Training Strategy	32
3.11.1 Global Model Pre-Training	32
3.11.2 Per-Plant Transfer Learning	32
3.11.3 Fine-Tuning Strategy	33
3.11.4 Training and Validation Procedure	34
3.12 Forecasting Workflow	34
3.12.1 15-Minute Forecasting Workflow	35
3.12.2 24-Hour Forecasting Workflow	35
3.12.3 96-Step Forecasting Workflow	35
3.13 Chapter Summary	36
Chapter 4: Experimental Results and Performance Evaluation	37
4.1 Introduction	37
4.2 Setup Discussion	37
4.2.1 Experimental Setup and Analysis	37
4.2.2 Evaluation Metrics	38
4.2.3 Implementation Environment and Toolkit	39

4.3 Forecasting Results	40
4.3.1 15-Minute Horizon Results	40
4.3.2 24-Hour Horizon Results	43
4.3.3 Per-Plant Transfer Learning Results	45
4.4 Comparison with Existing Approaches	47
4.4.1 Comparison with Random Forest	47
4.4.2 Comparison with XGBoost / Gradient Boosting	48
4.4.3 Comparison with ARIMA and MLP+TL (Reference Paper)	48
Chapter 5: Conclusions and Future Work	50
5.1 Conclusion	50
5.1.1 Summary of the Work	50
5.1.2 Key Findings — 15-Minute Horizon	50
5.1.3 Key Findings — 24-Hour Horizon	50
5.1.4 Key Findings — Transfer Learning	51
5.1.5 Random Forest vs. LSTM: An Unexpected Result	52
5.1.6 Achievement of the Stated Objectives	52
5.1.7 Scope Delivered Versus Scope Proposed	53
5.1.8 Limitations	53
5.2 Future Work	54
5.2.1 Methodological Corrections for a Fully Rigorous Comparison	54
5.2.2 Closing the Scope Gaps	55
5.2.3 Extended Benchmarking	55
5.2.4 Architectural and Training Extensions	56
5.2.5 Operational and Deployment Considerations	56
References	57

List of Figures

Figure 4.1	Actual-vs-predicted overlay plot for the 15-minute LSTM model on a representative plant 41
Figure 4.2	Top-12 feature importance bar chart, Random Forest 15-minute pipeline 41
Figure 4.3	Fleet-wide R^2 ranking bar chart, Random Forest 15-minute pipeline..... 42
Figure 4.4	Diurnal mean-absolute-error profile, Random Forest 15-minute pipeline..... 42
Figure 4.5	Actual-vs-predicted scatter plot (goodness of fit), Random Forest 15-minute pipeline..... 42
Figure 4.6	Actual-vs-predicted overlay plot, 24-hour LSTM model 43
Figure 4.7	Top-12 feature importance bar chart, Random Forest 24-hour pipeline..... 44
Figure 4.8	Fleet-wide R^2 ranking bar chart, Random Forest 24-hour pipeline 44
Figure 4.9	Diurnal mean-absolute-error profile, Random Forest 24-hour pipeline 44
Figure 4.10	Chronological 72-hour actual-vs-forecast window, Plant 47 45
Figure 4.11	Per-plant SMAPE comparison before and after Transfer Learning, 24-hour horizon 46
Figure 4.12	Training/validation loss curves for the global (base) LSTM model, 24-hour horizon 46

List of Tables

Table 3.1	Number of 15-minute time steps corresponding to each forecasting horizon.....	21
Table 4.1	Data partitioning and core hyperparameters per pipeline	37
Table 4.2	Software and hardware environment per model family	39
Table 4.3	Pooled test-set metrics, 15-minute horizon.....	40
Table 4.4	Per-plant paper-protocol metrics, 15-minute horizon	40
Table 4.5	Pooled test-set metrics, 24-hour horizon.....	43
Table 4.6	Per-plant paper-protocol metrics, 24-hour horizon.....	43
Table 4.7	Per-plant Transfer Learning outcome summary, both horizons.....	45
Table 4.8	Head-to-head comparison, Random Forest vs. LSTM (this work), both horizons ..	47
Table 4.9	XGBoost / Gradient Boosting benchmarks, reference dissertation [1]	48
Table 4.10	ARIMA and MLP(+TL) benchmarks, reference dissertation [1]	48
Table 5.1	Chapter 1 targets evaluated against the Chapter 4 results	52

Chapter 1: Introduction

1.1 Overview

The global energy landscape is undergoing a profound transformation, driven by the urgency of climate change, the depletion of fossil fuel reserves, and the steep decline in the cost of renewable energy technologies. Amid this shift, solar photovoltaic (PV) power has emerged as one of the fastest-growing sources of electricity generation worldwide. According to the International Energy Agency, solar PV capacity additions have consistently broken records in recent years, and the technology now represents a cornerstone of national energy transition strategies across both developed and developing economies.

Brazil presents a particularly compelling context for this transition. With one of the highest solar irradiance levels on Earth — especially across the northeastern and central-western regions — the country has experienced rapid growth in utility-scale PV installations over the past decade. The BR-PVGen dataset, which forms the foundation of this project, captures this reality through high-resolution measurements collected from 51 geographically distributed photovoltaic plants, recorded at 15-minute intervals across multiple years of operation.

Despite this potential, the large-scale integration of solar energy into modern power grids introduces a fundamental operational challenge: unlike conventional generation sources, solar power output is inherently intermittent and difficult to predict. The electricity a PV plant produces at any given moment is governed by atmospheric conditions — solar irradiance, cloud cover, temperature, wind, and humidity — which vary continuously and are not fully deterministic even hours ahead. Grid operators must balance electricity supply and demand in real time, and unexpected fluctuations in solar output require rapid compensating action through backup generation, energy storage dispatch, or demand-side management. The cost of these balancing operations, and the risk of grid instability they carry, grows significantly as the share of solar energy in the grid increases.

Accurate solar power forecasting is therefore not merely a technical convenience — it is a prerequisite for the reliable and economical operation of solar-heavy power systems. A forecast that correctly anticipates tomorrow's generation profile allows grid operators to schedule backup resources efficiently, reduces the need for costly spinning reserves, and enables better utilization of transmission infrastructure. At the plant level, accurate forecasts support maintenance scheduling, financial planning, and participation in electricity markets where advance generation commitments are required. It is this gap — between the operational value of accurate forecasting and the practical difficulty of achieving it at scale — that motivates the work undertaken in this project.

1.2 Motivation and Challenges

The motivation for this project arises from the convergence of two observations. First, despite significant advances in machine learning and deep learning over the past decade, the application of these techniques to solar power forecasting at scale — across many plants simultaneously, with heterogeneous characteristics and varying data quality — remains an open and practically important research problem. Second, the BR-PVGen dataset provides an unusually rich and realistic testbed for developing and evaluating such approaches, yet published work on this dataset has so far been limited primarily to classical machine learning methods.

The reference dissertation of de Jesus et al. [1], against which this project benchmarks its results, compared six models on the BR-PVGen dataset: ARIMA, Gradient Boosting, Random Forest, XGBoost, MLP, and MLP with Transfer Learning. While these models provide a solid baseline, they share an important limitation. ARIMA and the tree-based methods treat each timestep as an independent observation, failing to capture the sequential temporal dependencies that characterize solar generation — the way a cloud transient at 10:00 shapes the irradiance profile at 10:15, or the way yesterday's weather pattern informs today's forecast. MLP, while more flexible, operates on flat feature vectors and cannot natively model the dynamic temporal structure of the input signal either.

The challenges of solar power forecasting that motivated the design choices in this project can be summarized as follows.

Temporal complexity. Solar generation is governed by processes operating at multiple timescales simultaneously: the deterministic daily cycle of the sun's position, the stochastic variability of cloud cover on timescales of minutes to hours, and the seasonal evolution of irradiance over weeks and months. A model that captures only one of these timescales will systematically underperform at the others.

Plant heterogeneity. The 51 plants in the BR-PVGen dataset differ substantially in nominal capacity (ranging from small installations to large utility-scale farms), panel technology, mounting structure (fixed versus solar tracker), geographic location, and local microclimate. A single global model trained on all plants must generalize across this diversity, while a model trained on each plant individually is limited by that plant's available data — particularly for newer or smaller installations.

Data quality variation. Several plants in the dataset exhibit periods of anomalous readings caused by sensor failures, inverter faults, maintenance shutdowns, or grid curtailment events. During these periods, measured power output does not reflect the plant's actual generation capability, and including these samples in training without correction introduces misleading patterns that degrade model performance.

Forecast horizon difficulty. Predicting the next 15-minute power output is a fundamentally different task from predicting the output 24 hours ahead. The 15-minute task benefits from the strong autocorrelation of solar irradiance at short timescales — conditions rarely change dramatically in a single step. The 24-hour task requires the model to reason about what

tomorrow will look like based solely on today's observations, without access to any future weather measurements.

Taken together, these four challenges point toward a specific class of solution: a model expressive enough to capture temporal dependencies at multiple scales, trainable on the pooled fleet to overcome individual plants' data limitations, yet adaptable enough to specialize to each plant's local behavior. Section 1.3 formalizes this as a concrete problem statement, and Section 1.4 introduces the dual-input LSTM with per-plant Transfer Learning proposed to address it.

1.3 Problem Statement

This project addresses the following core problem:

Given a historical window of weather measurements and power output data from a photovoltaic plant — spanning 48 hours for the 24-hour-ahead task and 24 hours for the 15-minute-ahead task — together with static plant metadata describing its technical and geographic characteristics, can a deep learning model accurately forecast that plant's power output at two distinct horizons — 15 minutes ahead and 24 hours ahead — while generalizing effectively across 51 plants with heterogeneous characteristics and varying data quality?

More specifically, the project targets the following measurable objectives derived from the benchmark results of [1]:

For the 15-minute-ahead forecast, the target is to achieve a SMAPE lower than the best-performing model in [1] — MLP with Transfer Learning — which reported a mean SMAPE of 43.92% across 51 plants. This constitutes a direct and quantified benchmark for model quality.

For the **24-hour-ahead** forecast, no direct paper benchmark exists at this horizon. The targets adopted are a pooled R^2 above 0.80, NRMSE range below 15%, and SMAPE at peak hours below 20%, consistent with standards reported in the broader solar forecasting literature.

The problem is further constrained by practical considerations. The model must be capable of producing predictions for plants not seen during training — a realistic requirement for any system intended for deployment beyond the original 51 plants. It must handle plants with limited historical data gracefully, without catastrophic degradation. And the resulting system must be deployable as a production API capable of serving predictions in real time from incoming weather data.

1.4 Proposed Solution

To address the problem described above, this project proposes a dual-input Long Short-Term Memory (LSTM) neural network combined with a per-plant Transfer Learning strategy.

Dual-input LSTM architecture. The model receives two inputs simultaneously. The primary input is a sequential time-series window of historical observations at 15-minute resolution — 192 steps, covering 48 hours, for the 24-hour-ahead model, and 96 steps, covering 24 hours, for the 15-minute-ahead model — containing weather sensor measurements (plane-of-array irradiance, global horizontal irradiance, ambient temperature, panel temperature, wind speed, and wind direction) alongside derived temporal features (cyclic hour, day-of-year, and month

encodings, solar elevation angle at the target time, and clearness index). This sequence is processed by two stacked LSTM layers, which learn to extract temporal patterns at multiple scales. The secondary input is a static vector of seven plant metadata features (nominal power, panel count, efficiency, temperature coefficient, bifaciality coefficient, structure type, and geographic region), processed by a separate dense embedding branch. Both branches are concatenated and passed through a shared prediction head that produces a single normalized power output value.

Transfer Learning strategy. The model is trained in two stages. In the first stage, a global base model is trained on the pooled data from all 51 plants, allowing the LSTM layers to develop robust representations of the temporal dynamics common to all solar plants — irradiance ramps at sunrise and sunset, cloud transient patterns, and seasonal cycles. In the second stage, the LSTM layers are frozen and a separate copy of the model is fine-tuned for each individual plant using only that plant's own data. This fine-tuning adapts the dense prediction head to the plant's specific characteristics — its local shading patterns, inverter behavior, soiling rate, and operational history — without overwriting the general temporal knowledge acquired during global training. An adaptive learning rate strategy ensures that large plants with abundant data are fine-tuned conservatively, while small plants with limited sequences benefit from stronger adaptation.

Data quality handling. Plant 13, which exhibits persistent anomalous readings caused by maintenance-related outages, is excluded from the Transfer Learning fine-tuning stage and uses the global base model weights directly. For other plants, sequences containing invalid targets are silently skipped during sequence construction.

Production deployment. The trained model system is exposed through a FastAPI backend that accepts raw weather sequences and plant metadata, performs the complete feature engineering and scaling pipeline internally, selects the appropriate per-plant model, runs inference, de-normalizes the output, and returns the predicted power in kilowatts. The API supports both known plants from the training set and previously unseen plants using the global base model as a fallback.

1.5 Thesis Organization

The remainder of this report is organized as follows.

Chapter 2 — Background and Literature Review surveys the theoretical foundations and existing research relevant to this project. It opens with an overview of solar PV power forecasting, covering the distinction between intra-day and day-ahead forecast horizons, the principal challenges that make PV forecasting a difficult problem, and the role of Transfer Learning in time-series forecasting contexts. It then introduces the machine learning algorithms referenced throughout the report — including fundamental terminology, regression and time-series methods, Random Forest, Gradient Boosting and XGBoost, ARIMA, and Recurrent Neural Networks with particular focus on the LSTM architecture. The chapter concludes with a structured literature review of prior work on solar power forecasting, positioning this project within the current state of the art.

Chapter 3 — Proposed Dual-Horizon LSTM Forecasting Approach presents the complete technical specification of the proposed system. It begins by defining the two forecasting scenarios — the 15-minute-ahead horizon, which enables direct benchmark comparison with the reference paper, and the 24-hour-ahead horizon, which addresses a more operationally demanding task. The proposed workflow is then described end to end, followed by a detailed treatment of the BR-PVGen dataset including its data description, preparation pipeline, and handling of data quality issues such as sensor faults and maintenance gaps. The chapter then covers the feature engineering pipeline — temporal cyclic encoding, irradiance-derived features, and solar geometry at the target time — before presenting the dual-input model architecture comprising the LSTM time-series branch, the dense metadata branch, and the merged output layers. The chapter closes with the two-stage training strategy: Stage 1 global pre-training across all 51 plants, and Stage 2 per-plant Transfer Learning with frozen LSTM layers.

Chapter 4 — Experimental Results and Performance Evaluation reports the quantitative evaluation of the proposed system. It begins with the experimental setup and analysis, the evaluation metrics used throughout, and the implementation environment and software toolkit. Results are then presented separately for the 15-minute horizon and the 24-hour horizon, followed by a dedicated analysis of the per-plant Transfer Learning outcomes. The chapter closes with comparisons against the baseline models from the reference paper — Random Forest, XGBoost, Gradient Boosting, ARIMA, and MLP with Transfer Learning — across the shared evaluation metrics.

Chapter 5 — Conclusions and Future Work summarizes the contributions and findings of this project and proposes directions for future research and system improvement.

Chapter 2: Background and Literature Review

2.1 Introduction

The growing integration of solar photovoltaic (PV) systems into electrical grids has created a critical need for accurate power generation forecasting. Unlike conventional power sources, solar energy is inherently intermittent and dependent on atmospheric conditions that are difficult to predict, particularly over longer time horizons. Accurate forecasting enables grid operators to balance supply and demand efficiently, reduces the need for costly reserve capacity, and supports the economic viability of renewable energy investments — as established in Chapter 1.

This chapter provides the theoretical and empirical foundation for the research conducted in this project. Section 2.2 opens with an overview of solar PV power forecasting, covering the different forecasting horizons, the key challenges that make this problem difficult, and the role of Transfer Learning in improving model generalization across multiple plants. Section 2.3 then presents the machine learning concepts and algorithms relevant to this study, including Random Forest, XGBoost, ARIMA, and Long Short-Term Memory networks. Section 2.4 closes the chapter with a review of the existing literature on solar forecasting, with particular attention to the BR-PVGen dataset used in this project and the benchmark results established by prior research.

2.2 Solar PV Power Forecasting Overview

Solar PV power forecasting refers to the process of predicting the electrical output of a photovoltaic system at a future point in time, given information about the system's characteristics and the environmental conditions it will experience. The output of a PV system is primarily determined by the solar irradiance incident on the panel surface, the panel temperature, and the system's electrical efficiency. Since irradiance is driven by atmospheric dynamics — cloud cover, aerosols, humidity, and air mass — its future value is inherently uncertain, making PV forecasting a challenging regression problem.

Forecasting approaches are broadly categorized into three families based on their primary input type. Physical models use numerical weather prediction outputs and the physical equations governing solar geometry and panel operation to estimate generation. Statistical models learn the mapping between historical observations and future output from data, without encoding explicit physical knowledge. Machine learning models, which form the focus of this project, are a subclass of statistical approaches that use flexible, high-capacity function approximators to learn complex non-linear relationships from large datasets.

The choice of approach depends on the available data, the required forecasting horizon, and the computational resources available. For short horizons of minutes to hours, statistical and machine learning models tend to outperform physical models because the atmosphere changes slowly enough that recent observations provide strong predictive information. For horizons of multiple days, numerical weather prediction becomes more important because the local observational signal decays and large-scale atmospheric dynamics dominate.

2.2.1 Forecasting Horizons (Intra-day vs. Day-ahead)

Solar PV forecasting problems are classified by their temporal horizon — the gap between the time a forecast is made and the time it applies to. The four most commonly recognized horizons in the literature are very short-term (seconds to minutes), short-term or intra-day (minutes to hours), day-ahead (12 to 48 hours), and medium-term (days to weeks).

Intra-day forecasting targets horizons from several minutes to a few hours ahead. At these horizons, the most recent observations of irradiance, temperature, and power output carry strong predictive information because atmospheric conditions change gradually. Persistence models — which assume the next value will equal the current value — serve as a natural baseline at these horizons and are surprisingly competitive for horizons under 30 minutes. Machine learning models that incorporate lag features or sequential history typically outperform persistence for horizons from 15 minutes to 4 hours. The BR-PVGen dataset benchmark paper evaluated models at the 15-minute intra-day horizon, where the best-performing model, an MLP with Transfer Learning, achieved a mean SMAPE of 43.92% across 51 Brazilian plants.

Day-ahead forecasting targets a horizon of approximately 24 hours. This is the most operationally important horizon for electricity market participation, where generators must submit generation bids the day before delivery. At this horizon, the information content of recent observations is substantially reduced because the atmosphere can change completely within 24 hours — a clear morning provides little information about whether the following afternoon will be cloudy. This makes day-ahead forecasting fundamentally harder than intra-day forecasting, and models that rely heavily on persistence signals (as many machine learning models do through lag features) face a structural accuracy ceiling. This project targets the 24-hour-ahead horizon, which is more operationally valuable but inherently more challenging than the intra-day task studied in the benchmark paper.

The difficulty increase from intra-day to day-ahead can be understood through the concept of decorrelation time — the lag at which the autocorrelation of the power series drops to near zero. For most PV plants, this decorrelation time is between 2 and 6 hours, meaning that observations more than 6 hours old contribute very little linear predictive information about future output. A 24-hour forecast must therefore rely primarily on seasonal patterns, solar geometry, and weather forecast information rather than on recent power observations.

2.2.2 Challenges in PV Power Forecasting

Solar PV power forecasting is difficult for a combination of physical, statistical, and operational reasons. Understanding these challenges is essential for designing models that are robust in practice.

Atmospheric variability and cloud cover. The primary driver of forecast error at all horizons beyond a few minutes is the unpredictability of cloud cover. A cloud passing over a solar panel can reduce its output by 70–90% within seconds. At the 24-hour horizon, the state of the atmosphere changes too rapidly for recent observations to constrain next-day cloudiness, and numerical weather predictions are themselves imperfect at the spatial and temporal scales relevant to individual plants. Studies consistently find that 60–80% of total forecast error in

day-ahead PV forecasting can be attributed to cloud-related uncertainty, with the remainder driven by temperature effects, aerosol loading, and instrumentation noise. This means that no model architecture, however sophisticated, can achieve very low errors at the 24-hour horizon without access to accurate weather forecast inputs.

Nonlinearity and threshold effects. The relationship between irradiance and power output is nonlinear. At low irradiance levels, panels operate below their rated efficiency due to the nonlinear current-voltage characteristic of photovoltaic cells. At very high irradiance under clear-sky conditions, inverter clipping limits output at the nominal AC power rating. The panel temperature also introduces a nonlinear efficiency loss, typically around 0.3–0.4% per degree Celsius above the standard test condition temperature of 25°C. These threshold effects mean that simple linear models consistently underperform at both extremes of the operating range.

Sunrise and sunset transitions. Power output transitions sharply from zero to nonzero at sunrise and back to zero at sunset. These transitions create steep gradients in the power time series that are difficult to predict precisely because small errors in the predicted timing of sunrise or in the early-morning irradiance can produce large absolute errors. The SMAPE metric is particularly sensitive to these transition periods because both the actual and predicted values are small, making the denominator small and the percentage error large. Many forecasting studies filter these transition periods out of their error calculations, though this is not universal.

Inter-plant variability. A dataset of multiple plants introduces the challenge of heterogeneity. Plants differ in nominal power capacity (1 to 5 MW in the BR-PVGen dataset), panel technology and efficiency, orientation and tilt, tracking versus fixed structure, and geographic location spanning multiple Brazilian states with different climatic regimes. A model trained jointly on all plants must learn to generalize across this variability, while a model trained separately on each plant must manage with less data. The BR-PVGen dataset has plants with as few as 11,520 rows (approximately 4 months) alongside plants with over 42,000 rows (nearly 3 years). Low-data plants are particularly difficult to model because the train-test split may place the test period in a season not represented in training.

Data quality issues. Real-world sensor data contains gaps, sensor faults, clipping artefacts from inverter saturation, and periods of plant unavailability due to maintenance or grid outages. The BR-PVGen dataset documentation identifies unavailability losses of 6.6% and sensor-related losses categorized under "others" at 9.6%. Plants with high unavailability rates present a challenge because the power-irradiance relationship during outage periods is structurally different from normal operation, and a model trained without distinguishing these periods learns a corrupted mapping. Plant 13 (PS_014) in the BR-PVGen dataset is a notable example: with only approximately four months of operational history against more than a year for most other plants in the fleet, it is excluded from the Transfer Learning fine-tuning stage in this project, and in evaluations where it is included — such as the Random Forest models — it is consistently the lowest-performing plant. This illustrates how limited data volume alone, independent of any sensor-level corruption, can produce systematically weaker model fit.

Temporal non-stationarity. PV system performance degrades gradually over time due to panel soiling, degradation of module efficiency, and ageing of inverter components. A model

trained on data from year 1 may systematically overpredict output in year 3 if this degradation is not accounted for. Seasonal non-stationarity is also significant — the solar resource in Brazil varies considerably between summer and winter, and models trained predominantly on one season generalize poorly to the other. This is why this project uses strict chronological train-test splits rather than random sampling, ensuring that the model is always evaluated on data that is later in time than its training data.

The curse of dimensionality in feature engineering. Lag-feature-based models for 24-hour-ahead forecasting require features derived from observations 96 or more timesteps in the past. Adding lags for multiple variables (power, irradiance, temperature) at multiple time offsets quickly creates a high-dimensional feature space. Many of these lag features are highly correlated, which can destabilize tree-based models through feature redundancy and slows convergence in neural networks through the associated ill-conditioned gradient landscape.

2.2.3 Transfer Learning in Time-Series Forecasting

Transfer Learning is a machine learning paradigm in which knowledge acquired during training on a source task or dataset is reused to improve performance on a related target task or dataset. The motivation is straightforward: when the target dataset is small or the target task is similar to the source task, training from scratch on the target alone is inefficient and prone to overfitting, while incorporating the source task's learned representations provides a regularizing prior that improves generalization.

In the context of deep learning, Transfer Learning typically operates by pre-training a model on a large source dataset, then fine-tuning it on a smaller target dataset. The fine-tuning process usually involves freezing the early layers of the network — which are assumed to have learned general, transferable representations — while allowing the later layers to be updated on the target data. The rationale is that early layers in deep networks learn low-level patterns (in time-series forecasting, these correspond to general temporal dynamics such as periodicity and trend) while later layers learn task-specific patterns.

For solar PV forecasting across multiple plants, Transfer Learning is particularly well-motivated. All PV plants share the same underlying physics — they all respond to the same solar geometry, the same basic irradiance-to-power conversion mechanism, and the same temperature coefficient relationship. The differences between plants are in their scale (nominal power), their efficiency and technology, their tracking configuration, and their local microclimate. A model pre-trained across all plants learns the shared physics, and fine-tuning adapts it to the plant-specific differences.

The BR-PVGen benchmark paper demonstrated this principle empirically with MLP models. Pre-training on the plant with the most data (Plant 1) and fine-tuning on each target plant reduced mean SMAPE from 47.24% to 43.92% — a reduction of 7.02% — across all 51 plants at the 15-minute horizon. The gain was larger for plants with fewer records: 11.06% SMAPE reduction for plants with fewer than 4,800 records, compared to only 3.81% for the full dataset. This confirms the theoretical expectation that Transfer Learning is most valuable precisely in the data-limited scenarios where models would otherwise overfit.

This project implements Transfer Learning on the LSTM architecture. The global LSTM model is trained across all 50 plants simultaneously, learning shared temporal and physical patterns. The per-plant fine-tuning then freezes the LSTM layers — which encode the shared temporal dynamics — and updates only the dense output head, which adapts to the plant-specific power scale and operating characteristics. This design choice follows the logic of the BR-PVGen paper's Transfer Learning implementation but extends it from MLP to LSTM, which has the additional advantage of explicitly modelling the temporal order of observations through its recurrent hidden state.

2.3 Machine Learning Overview

Machine learning is a field of artificial intelligence concerned with the development of algorithms that learn from data to perform tasks without being explicitly programmed for each scenario. Rather than encoding domain knowledge as hand-crafted rules, machine learning systems identify patterns in historical examples and use these patterns to make predictions or decisions on new data. In the context of solar PV forecasting, machine learning models learn the mapping from observed weather and historical power data to future power output by optimizing their parameters to minimize prediction error on a labelled training dataset.

The success of machine learning for PV forecasting stems from several properties. Machine learning models can represent highly nonlinear relationships that analytical models would need to approximate with simplified assumptions. They can automatically identify which input features are most informative without requiring domain expertise to specify these relationships. They scale well with data volume — given enough training examples, modern machine learning models continue to improve in a way that physics-based models do not. And they can be retrained efficiently as new data becomes available, making them well-suited to operational deployment.

2.3.1 Fundamental Terms of Machine Learning

Several foundational concepts appear throughout the machine learning literature and throughout this project.

Supervised learning is the paradigm in which a model is trained on labelled examples — input-output pairs — to learn a function that maps inputs to outputs. PV power forecasting is a supervised learning problem: the inputs are weather observations and historical power data, and the output is future power generation.

Training, validation, and test sets are three non-overlapping subsets of the data serving distinct purposes. The training set is used to fit model parameters. The validation set is used during training to monitor generalization and tune hyperparameters without using the test set. The test set is held out entirely until after all modelling decisions are made and is used only to report final performance. In time-series forecasting, these splits must be made chronologically — the training set contains earlier observations, and the test set contains later ones — to avoid data leakage.

Overfitting occurs when a model learns the specific noise patterns in the training data rather than the underlying signal, resulting in poor generalization to unseen data. It is detected by a large gap between training and validation performance. Common remedies include regularization, dropout, early stopping, and using more training data.

Regularization refers to techniques that constrain the model's complexity to prevent overfitting. L2 regularization adds a penalty proportional to the squared magnitude of model weights to the loss function, discouraging large weights. Dropout randomly zeroes a fraction of neural network activations during training, preventing co-adaptation of neurons. Early stopping terminates training when validation performance stops improving, using the model from the best validation epoch.

Hyperparameters are model configuration choices that are not learned from data but must be set before training — for example, the number of trees in a Random Forest, the learning rate in a neural network, or the sequence length for an LSTM. Hyperparameter selection is typically done by evaluating performance on the validation set across a range of candidate values.

Loss functions define what the model optimizes during training. Mean Squared Error penalizes large errors more than small ones due to the squaring operation. Mean Absolute Error treats all errors proportionally. Huber loss combines these properties — it is quadratic for small errors and linear for large errors, controlled by a threshold parameter δ . This project uses Huber loss with $\delta = 0.15$ to balance sensitivity to large errors with robustness to occasional extreme outliers.

2.3.2 Regression and Time-Series Algorithms

Time-series forecasting is a specialized regression problem in which the goal is to predict future values of a sequence based on its past values and potentially other covariates. The key distinguishing property of time-series data is temporal dependence — observations close in time are correlated, and the order of observations carries information. Algorithms for time-series forecasting must either explicitly model this temporal dependence or incorporate it through feature engineering.

Two broad strategies are used. Autoregressive approaches model the current value as a function of past values of the same series. They include classical statistical models such as ARIMA and modern machine learning adaptations using lag features. Sequence-modelling approaches process the entire historical window as a structured input, learning temporal dependencies implicitly. They include Recurrent Neural Networks, Long Short-Term Memory networks, and Transformer-based architectures.

2.3.3 Random Forest Algorithm

Random Forest is an ensemble learning algorithm introduced by Breiman [2] that constructs a large number of decision trees during training and outputs the average of their individual predictions for regression tasks. Each tree is built on a randomly drawn bootstrap sample of the training data, and at each node split, only a random subset of features is considered as candidates. This double randomization — in the data and in the features — decorrelates the individual trees, so their errors are partly independent and average out in the ensemble.

The key properties of Random Forest relevant to this project are its robustness to irrelevant features, its ability to model nonlinear relationships without explicit feature transformation, and its native resistance to overfitting due to the averaging across many trees. Random Forest does not require feature scaling and handles missing values gracefully. Its main limitation for time-series forecasting is that it treats each training sample as independent — it has no mechanism for modelling the temporal order of observations. This means that temporal dependencies must be encoded explicitly through lag features, which requires careful design to avoid data leakage.

In the BR-PVGen benchmark, Random Forest achieved a mean SMAPE of 47.33% and NRMSE of 13.82% across 51 plants at the 15-minute horizon, placing it in the middle tier of the evaluated models. The relatively high SMAPE despite the competitive NRMSE reflects Random Forest's known weakness in predicting extreme values — it cannot extrapolate beyond the range of training targets because tree predictions are bounded by the training labels.

2.3.4 Gradient Boosting / XGBoost Algorithm

Gradient Boosting is an ensemble algorithm that builds trees sequentially rather than in parallel. Each new tree is trained to predict the residual errors of the current ensemble, with the ensemble's prediction updated by adding the new tree's output multiplied by a learning rate. This sequential error-correction process allows Gradient Boosting to achieve lower bias than Random Forest at the cost of higher variance and greater sensitivity to overfitting, which is mitigated by the learning rate and tree depth hyperparameters.

XGBoost (Chen and Guestrin, 2016) is a computationally optimized implementation of Gradient Boosting that introduced second-order gradient information into the tree-building criterion, regularization terms directly in the objective function, and efficient parallel computation. These innovations made XGBoost substantially faster and more accurate than earlier Gradient Boosting implementations and led to its widespread adoption for tabular data competitions and industrial applications.

For PV forecasting, XGBoost shares the same fundamental limitation as Random Forest regarding temporal dependence — it operates on flat feature vectors and requires lag engineering to incorporate history. However, XGBoost is often more data-efficient than Random Forest and handles the interaction between features more effectively due to the boosting mechanism. In the BR-PVGen benchmark, XGBoost achieved a mean SMAPE of 46.48% — the best among the classical machine learning models — with the lowest standard deviation ($\pm 8.13\%$), indicating more consistent performance across plants.

2.3.5 ARIMA Algorithm

ARIMA — Auto Regressive Integrated Moving Average — is a classical statistical time-series model introduced by Box and Jenkins [3]. It decomposes the forecasting problem into three components: the autoregressive component $AR(p)$, which models the current value as a linear combination of p past values; the integrated component $I(d)$, which applies d -order differencing to remove trend and make the series stationary; and the moving average component $MA(q)$, which models the current value as a linear combination of q past forecast errors.

ARIMA is a univariate model — it uses only the historical values of the target series itself, without incorporating exogenous variables such as irradiance or temperature (though the extended ARIMAX model allows exogenous inputs). This is a significant limitation for PV forecasting, where the primary drivers of output — irradiance and temperature — are exogenous to the power series. ARIMA's forecasts at long horizons tend to revert rapidly to the series mean, producing flat predictions that poorly capture the diurnal solar cycle beyond a few hours.

Despite these limitations, ARIMA serves as an important baseline in the literature because it requires minimal data and no hyperparameter tuning beyond the selection of p , d , and q . In the BR-PVGen benchmark, ARIMA achieved a mean SMAPE of 47.05% at the 15-minute horizon — comparable to Random Forest and XGBoost, reflecting the fact that at very short horizons, the strong autocorrelation of the power series allows simple univariate models to compete with more complex approaches.

2.3.6 Recurrent Neural Networks and LSTM

Recurrent Neural Networks (RNNs) are a class of neural networks designed explicitly for sequential data. Unlike feedforward networks, which process inputs independently, RNNs maintain a hidden state — a vector that summarizes information from all previous timesteps — and update it at each new input. The prediction at any timestep is a function of the current input and the hidden state, allowing the network to model dependencies across arbitrary time lags.

The fundamental challenge with standard RNNs is the vanishing gradient problem. During backpropagation through time, gradients are multiplied by the recurrent weight matrix at each timestep. If this matrix has singular values less than one, the gradient shrinks exponentially with the number of timesteps, making it impossible for the network to learn dependencies spanning more than a few steps. This effectively limits standard RNNs to short-range temporal dependencies.

Long Short-Term Memory (LSTM) networks, introduced by Hochreiter and Schmidhuber [4], solve the vanishing gradient problem through a gating mechanism. An LSTM cell maintains two state vectors: a hidden state h and a cell state c . The cell state acts as a long-term memory that can carry information across hundreds of timesteps with minimal gradient decay, because information is added and removed from it through additive operations controlled by three gates: the input gate, the forget gate, and the output gate. Each gate is a sigmoid-activated linear function of the current input and the previous hidden state, producing values between 0 and 1 that act as soft switches controlling information flow.

The forget gate decides what fraction of the previous cell state to retain. The input gate decides what new information from the current input to add to the cell state. The output gate determines what fraction of the cell state to expose as the hidden state. This architecture allows LSTMs to selectively remember relevant past information — for example, the irradiance pattern from the same hour of the previous day — while discarding irrelevant information.

For PV forecasting, LSTMs offer several structural advantages over tabular machine learning models. They process the raw time series directly without requiring explicit lag feature

engineering, allowing the network to discover which past timesteps are most informative. They can model complex temporal dependencies including seasonality, trend, and cross-variable interactions across the lookback window used in this project — 192 steps (48 hours) for the 24-hour-ahead model and 96 steps (24 hours) for the 15-minute-ahead model. They support the integration of static metadata — plant specifications — through a parallel input branch that is concatenated with the LSTM output before the prediction head, enabling a single architecture to generalize across plants with different capacities and configurations.

The dual-branch LSTM architecture used in this project processes the time-series features through two stacked LSTM layers and the static plant metadata through two dense layers, merging the outputs through concatenation before the final prediction. This architecture is trained globally across all plants and then fine-tuned per-plant using Transfer Learning, combining the data-efficiency benefits of shared learning with the accuracy benefits of plant-specific adaptation.

2.4 Literature Review

The forecasting of solar PV power output has been extensively studied over the past two decades, with a substantial body of work comparing classical statistical methods, machine learning approaches, and deep learning architectures across a range of datasets and horizons.

Early work on statistical approaches established ARIMA and its seasonal variant SARIMA as standard baselines for short-term PV forecasting. Mellit and Pavan [5] demonstrated that feedforward neural networks outperformed ARIMA for day-ahead forecasting by capturing the nonlinear relationship between meteorological inputs and power output, establishing a pattern that would be repeated across subsequent comparative studies. Bacher et al. [6] showed that adaptive online learning methods could further improve on static models by continuously updating model parameters as new observations became available, highlighting the importance of temporal non-stationarity.

The emergence of ensemble methods brought substantial improvements for tabular PV forecasting. Ensemble approaches combining Random Forest, Gradient Boosting, and Support Vector Regression have been reported to outperform single models for intra-day solar forecasting in the broader literature, with Random Forest often showing robust performance across diverse climatic conditions. The explanation typically given is the ensemble's ability to average out individual tree errors and its implicit feature selection through random feature sampling at each split. [A specific supporting citation for this paragraph was not confirmed during preparation of this chapter and should be added once verified — see Section 2.4 note.]

Deep learning approaches gained traction in solar forecasting following the broader success of LSTMs in sequence modelling tasks. LSTM networks have been applied to multi-step-ahead solar irradiance forecasting in several studies, with reported gains over classical statistical and shallow neural network baselines attributed to the LSTM's ability to learn diurnal periodicity without explicit seasonal feature engineering; other studies report convolution-based architectures matching or exceeding LSTM performance for the same task, so this remains an active area rather than a settled comparison. [Specific supporting citations for this paragraph

were not confirmed during preparation of this chapter and should be added once verified — see Section 2.4 note.]

The application of Transfer Learning to PV forecasting was motivated by the practical challenge of deploying forecasting models for new plants with limited historical data. Transfer learning from data-rich source plants to data-limited target plants has been reported in the literature to improve forecasting accuracy, with larger gains typically found for target plants sharing geographic proximity and similar panel technology with the source plant — a plant-similarity-based transfer paradigm consistent with the approach adopted in this project (see Section 3.11.2). [A specific supporting citation for this paragraph was not confirmed during preparation of this chapter and should be added once verified — see Section 2.4 note.]

The most directly relevant prior work for this project is the dissertation and accompanying dataset paper by de Jesus et al. [1], which established the BR-PVGen dataset and its benchmark results — the same 51-plant Brazilian PV dataset used in this project. That work evaluated 12 models spanning classical statistical methods (ARIMA), linear models (Ridge, Lasso), tree-based ensembles (Random Forest, Gradient Boosting, XGBoost, Bagging), kernel methods (SVR), and neural networks (MLP, KNN, Decision Tree, AdaBoost) at both daily and 15-minute intra-day forecasting horizons. The MLP with Transfer Learning achieved the best overall performance at the 15-minute horizon, with a mean SMAPE of 43.92% and NRMSE of 14.07% across all 51 plants. The Transfer Learning experiment used the plant with the most historical data as the pre-training source, froze the first two layers during fine-tuning, and demonstrated that gains were largest for plants with fewer than 4,800 records — validating the hypothesis that transfer learning is most valuable in data-limited settings.

This project extends the BR-PVGen benchmark in three significant ways. First, it targets the 24-hour-ahead forecasting horizon rather than the 15-minute intra-day horizon, addressing the operationally more important day-ahead market participation use case. Second, it introduces an LSTM architecture not evaluated in the benchmark study, replacing the benchmark's flat feature vector input with a sequential window of raw observations — 48 hours for the 24-hour-ahead model and 24 hours for the 15-minute-ahead model — that allows the model to learn temporal dependencies directly from the raw time series. Third, it incorporates a dedicated plant metadata branch — processing nominal power, panel efficiency, tracking configuration, temperature coefficient, and geographic state through a parallel neural network branch — which is architecturally designed to support predictions for previously unseen plants given only their specifications and a weather forecast, a capability the flat, plant-specific benchmark models do not support by design. Whether this generalization capability holds empirically for plants entirely absent from training has not yet been tested in this work and is identified as a priority item for future validation (Section 5.2).

The dual-branch LSTM with Transfer Learning achieves an overall R^2 of 0.790 on the held-out test set across 50 plants, with Transfer Learning improving the validation loss for 25 out of 50 plants compared to the global base model. The comparison with the benchmark must account for the horizon difference — 24-hour-ahead forecasting is substantially harder than 15-minute-ahead — but this does not yet translate into a demonstrated advantage: as reported in Chapter 4, the LSTM's per-plant SMAPE (48.1–48.5%, depending on Transfer Learning)

remains higher than the reference benchmark's best 15-minute result (43.92%) even before adjusting for the harder horizon, and a rigorous, quantitatively controlled comparison across horizons has not yet been performed. The metadata branch's architectural capacity to accept previously unseen plant specifications is a structural difference from the benchmark's plant-by-plant training paradigm, but — as noted above — this capability has not yet been empirically validated and should not yet be presented as a demonstrated advancement.

CHAPTER 3:

PROPOSED DUAL-HORIZON SOLAR POWER FORECASTING APPROACH

3.1 Introduction

This chapter presents the proposed methodology for forecasting photovoltaic (PV) power generation using historical power-generation and meteorological data. The main objective of the proposed system is to estimate future solar power generation at different forecasting horizons while maintaining a methodology that can be applied across multiple photovoltaic plants.

The proposed approach considers two main forecasting horizons: short-term forecasting at a 15-minute horizon and longer-term forecasting at a 24-hour horizon. In addition, a 24-hour multi-step forecasting scenario is considered, in which the 24-hour period is represented by 96 consecutive time steps because the dataset has a 15-minute sampling interval.

Two machine-learning approaches are investigated: a Long Short-Term Memory (LSTM) neural network and a Random Forest (RF) regression model. The LSTM model is designed to exploit temporal relationships in sequential observations, while Random Forest provides a tree-based regression approach for learning the relationship between the input variables and future PV power generation.

The methodology also incorporates transfer learning into the LSTM forecasting approach. A global model is first trained using data from multiple photovoltaic plants. The learned model is then adapted to individual plants through plant-specific fine-tuning. This strategy is particularly useful when the amount of historical data available for a particular plant is limited.

The chapter describes the forecasting scenarios, dataset, data preparation, feature engineering, input-output configuration, proposed models, training strategy, and final forecasting workflow.

3.2 Proposed Forecasting Scenarios

The proposed system evaluates PV power forecasting at different temporal horizons. The dataset is sampled at 15-minute intervals, which means that four observations are available for every hour and 96 observations represent a complete 24-hour period.

The forecasting scenarios are designed to evaluate both short-term and day-ahead prediction capabilities.

The proposed scenarios are:

1. **15-minute-ahead forecasting**
2. **24-hour-ahead forecasting**
3. **24-hour multi-step forecasting using 96 steps**

These scenarios allow the proposed models to be evaluated under both immediate short-term forecasting and longer forecasting conditions.

3.2.1 15-Minute-Ahead Forecasting

The first forecasting scenario predicts the PV power generation at the next 15-minute time step.

If the current observation occurs at time t , the target is:

$$\hat{P}_{t+15}$$

where:

- P represents PV power generation.
- t represents the current time.
- $t + 15$ represents the next 15-minute time step.
- $\hat{P}$ represents the predicted power output.

The purpose of this scenario is to evaluate the ability of the models to capture short-term variations in PV generation.

Short-term PV power generation can change rapidly due to changes in solar irradiance and weather conditions. Therefore, this forecasting scenario provides a useful test of the model's ability to learn local temporal patterns.

The 15-minute scenario is implemented using both the LSTM and Random Forest models.

3.2.2 24-Hour-Ahead Forecasting

The second forecasting scenario considers a longer forecasting horizon of 24 hours.

Because the dataset has a 15-minute sampling interval:

$$\text{Steps per hour} = \frac{60}{15} = 4$$

and therefore:

$$\text{24-hour horizon} = 24 \times 4 = 96$$

Thus, a 24-hour-ahead target corresponds to 96 fifteen-minute intervals.

For a specific timestamp t , the objective is to predict the PV power corresponding to the same future time on the following day:

$$\hat{P}_{t+24h}$$

Thus, a 24-hour-ahead target corresponds to 96 fifteen-minute intervals.

For a specific timestamp t , the objective is to predict the PV power corresponding to the same future time on the following day:

$$P^{t+24h} \hat{P}_{t+24h}$$

For example, if the current observation corresponds to 10:00 AM, the model can be configured to forecast the power at 10:00 AM on the following day.

This forecasting scenario is particularly important for planning and operational decision-making because the model must learn longer-term relationships between historical power generation and weather-related variables.

The 24-hour forecasting experiment is conducted using both LSTM and Random Forest.

3.2.3 24-Hour Multi-Step Forecasting (96 Steps)

In addition to the 24-hour-ahead prediction, the proposed system considers a multi-step forecasting scenario in which the complete following 24-hour period is predicted.

Since each observation represents 15 minutes:

$$96 \times 15 = 1440 \text{ minutes} = 24 \text{ hours}$$

Therefore, the model produces 96 future forecasting steps covering the complete 24-hour period.

Conceptually, the output sequence can be represented as:

$$[\hat{P}_{t+15}, \hat{P}_{t+30}, \hat{P}_{t+45}, \dots, \hat{P}_{t+1440}]$$

This scenario is different from predicting only one point at $t+24t+24t+24$ hours. Instead, it represents a complete predicted PV power profile over the following day.

The 96-step forecasting scenario provides a more comprehensive representation of the expected PV generation profile and can be useful for applications such as energy management and operational planning.

3.3 Proposed System Workflow

The proposed forecasting system follows a sequence of processing stages beginning with historical PV generation and environmental data and ending with predicted future power generation.

The overall workflow consists of the following stages:

1. Dataset collection
2. Data loading and integration
3. Date-time processing
4. Data cleaning
5. Data splitting
6. Feature engineering
7. Input sequence construction
8. Model training
9. Global LSTM pre-training
10. Plant-specific transfer learning
11. Random Forest training
12. Forecast generation
13. Model evaluation

The historical data are first prepared and converted into a suitable format for machine-learning models. Temporal and meteorological features are then combined with historical power information.

The LSTM model receives sequential observations and learns temporal dependencies within the historical data. Random Forest uses the constructed feature representation to learn the relationship between the input features and the target power output.

For the LSTM approach, a global model is first trained and subsequently fine-tuned for individual plants. This enables the model to learn general PV generation patterns while also adapting to plant-specific behavior.

The final predictions are evaluated using appropriate regression and forecasting metrics.

3.4 BR-PVGen Dataset

3.4.1 Dataset Description

The proposed system uses the BR-PVGen dataset as the main source of photovoltaic generation information. The dataset contains data from multiple photovoltaic plants and is suitable for investigating PV power forecasting across different plants.

The dataset is particularly relevant to the proposed methodology because it allows the models to learn from multiple plants rather than relying exclusively on the historical data of one installation.

The reference dissertation of de Jesus et al. [1] reports experiments involving 51 photovoltaic plants and evaluates forecasting models individually for each plant before consolidating the results across the complete group.

The use of multiple plants also supports the proposed transfer-learning strategy. A model can learn general relationships from the available plants and subsequently be adapted to a specific target plant.

3.4.2 Solar Power Generation Data

The main target variable in the proposed forecasting system is the generated PV power.

Historical power-generation measurements provide information about how the photovoltaic plant behaves over time. These observations allow the models to learn patterns such as:

- increases in generation during periods of increasing solar irradiance;
- reduction of generation during periods of low irradiance;
- daily generation patterns;
- variations caused by weather conditions;
- plant-specific generation behavior.

The historical power values are also used as temporal information for the LSTM model.

The target variable is the future PV power generation corresponding to the selected forecasting horizon.

Therefore, the fundamental prediction relationship can be expressed as:

$$\text{Historical Power} + \text{Weather Information} \rightarrow \text{Future PV Power}$$

3.4.3 Weather and Environmental Data

Meteorological and environmental variables are important inputs because solar power generation is strongly affected by atmospheric and solar conditions.

The forecasting system uses weather-related information available in the dataset. Depending on the specific dataset columns and preprocessing implementation, these variables include parameters related to solar irradiance and environmental conditions.

Important variables considered by the forecasting methodology include:

- solar irradiance;
- temperature;
- humidity;
- wind-related measurements;
- other available environmental measurements.

Solar irradiance is particularly important because PV generation is directly related to the amount of solar radiation reaching the photovoltaic modules.

de Jesus et al. [1] also identifies irradiance and temperature as important plant-specific patterns during transfer learning.

3.4.4 Data Structure and Sampling Interval

The dataset is organized as time-series observations. Each observation corresponds to a specific timestamp and contains information about the photovoltaic plant and its associated measurements.

The forecasting system uses a **15-minute sampling interval**.

Therefore:

4 observations/hour

and:

96 observations/day

This sampling frequency is fundamental to the design of the forecasting horizons.

For example:

Table 3.1 Number of 15-minute time steps corresponding to each forecasting horizon

Forecasting task	Number of steps
15 minutes	1
1 hour	4
6 hours	24
12 hours	48
24 hours	96

This structure allows the same dataset to support both short-term and day-ahead forecasting experiments.

3.5 Data Preparation

Data preparation is required before the dataset can be provided to the forecasting models. The objective of this stage is to ensure that the observations are correctly ordered in time, that the relevant variables are available, and that the data are represented in a form suitable for machine-learning algorithms.

3.5.1 Data Loading and Integration

The first stage consists of loading the available data and organizing the observations according to the photovoltaic plant and timestamp.

For a multi-plant forecasting problem, the plant identifier is retained so that the model can distinguish observations originating from different photovoltaic plants.

The data are then organized chronologically.

The general structure can be represented as:

$$D = \{P_i, t_i, X_i\}$$

where:

- P_i is the measured power;
- t_i is the timestamp;
- X_i represents the available environmental and temporal features.

The resulting dataset provides the basis for both LSTM sequence construction and Random Forest feature construction.

3.5.2 Date-Time Processing and Time Alignment

The timestamp is an essential component of the forecasting problem because the observations represent a time-dependent sequence.

The data are therefore sorted chronologically before constructing the forecasting inputs.

Time alignment is particularly important when combining power-generation and environmental information. The weather and power measurements associated with the same timestamp must correspond to the same observation.

After determining the sampling frequency, the forecasting horizon can be expressed in terms of the number of time steps.

For a 15-minute interval:

$$\text{Steps per hour} = \frac{60}{15} = 4$$

and:

$$\text{24-hour horizon} = 24 \times 4 = 96$$

This relationship is used throughout the forecasting experiments.

3.5.3 Handling Missing and Invalid Values

Real-world photovoltaic datasets can contain missing, invalid, or abnormal observations.

Such values can negatively affect the training process and can lead to incorrect relationships being learned by the forecasting models.

Therefore, the data preparation stage includes identifying and handling invalid or missing observations according to the preprocessing procedure implemented in the project.

Special attention is required for PV power values because zero generation can have different meanings. For example, a zero value can represent nighttime conditions, but it can also potentially occur because of plant unavailability or measurement problems.

de Jesus et al. [1] also emphasizes that outliers and measurement failures can occur in photovoltaic plant data, which is one reason robust training mechanisms such as Huber loss were considered in the associated neural-network methodology.

3.5.4 Data Splitting

The prepared dataset is divided into training, validation, and testing portions.

The training data are used to learn the model parameters.

The validation data are used during model development to monitor generalization and support decisions such as early stopping and learning-rate adjustment.

The testing data are reserved for the final evaluation of forecasting performance.

For time-series forecasting, the temporal ordering of the observations must be preserved. Future observations should not be used to train the model when evaluating its ability to forecast those future observations.

This is important because randomly mixing observations from different points in time can result in information leakage and unrealistic evaluation results.

3.5.5 Feature Scaling and Normalization

Feature scaling is applied where required to ensure that variables with different numerical ranges do not disproportionately influence the learning process.

This is especially important for the LSTM model because neural networks can be sensitive to differences in feature magnitude.

The scaling process is fitted using the training data and then applied consistently to the validation and testing data.

The same transformation must be retained for the prediction stage so that new observations are presented to the trained model using the same feature representation used during training.

After prediction, the power output can be transformed back to its original scale for interpretation and reporting.

3.6 Feature Engineering

Feature engineering transforms the raw measurements into variables that provide useful information to the forecasting models.

The proposed feature set combines temporal information, environmental information, solar irradiance information, and historical power-generation information.

This combination allows the models to learn both the daily temporal pattern of PV generation and the influence of environmental conditions.

3.6.1 Temporal Features

Temporal features are derived from the timestamp.

Examples include:

- hour of the day;
- day of the year;
- month;
- day of the week;
- weekend indicator.

These variables provide information about the position of each observation within the daily and annual cycles.

For example, the hour of the day provides information about whether an observation occurs around sunrise, midday, afternoon, or sunset.

This is useful because PV generation has a strong relationship with the position of the sun throughout the day.

3.6.2 Cyclic Time Encoding

Directly representing time using integer values can incorrectly imply that the beginning and end of a cycle are far apart.

For example, hour 23 and hour 0 are adjacent in time even though their numerical values are far apart.

To represent this periodic behavior, cyclic encoding can be used.

For hour h :

$$\begin{aligned} hour_{sin} &= \sin\left(\frac{2\pi h}{24}\right) \\ hour_{cos} &= \cos\left(\frac{2\pi h}{24}\right) \end{aligned}$$

Similarly, the day of the year can be represented using sine and cosine transformations.

This allows the model to learn the cyclic nature of solar generation patterns.

3.6.3 Solar Irradiance Features

Solar irradiance represents one of the most important variables for PV power forecasting.

The model uses available irradiance information to establish the relationship between solar radiation conditions and the expected power output.

The irradiance-related features help the model distinguish between conditions such as:

- low solar radiation;
- increasing radiation;
- high radiation around midday;
- decreasing radiation;
- nighttime conditions.

The importance of irradiance is also reflected in de Jesus et al. [1], which considers irradiance and temperature among the plant-specific patterns captured during transfer learning.

3.6.4 Historical Power Features

Historical power generation is used to provide information about the previous behavior of the photovoltaic plant.

Previous power measurements can contain useful temporal information that may not be fully represented by weather variables alone.

Examples include:

$$P_{t-1}$$

$$P_{t-2}$$

and longer historical observations.

These values help the model identify the current generation state and recent generation trend.

For the LSTM model, historical information is naturally represented as a sequence. For Random Forest, historical values can be represented as separate tabular features.

3.6.5 Target Variable Construction

The target variable represents the future PV power generation that the model is required to predict.

For the 15-minute scenario:

$$y_t = P_{t+1}$$

where one step corresponds to 15 minutes.

For the 24-hour-ahead scenario:

$$y_t = P_{t+96}$$

because:

$$96 \times 15 \text{ minutes} = 24 \text{ hours}$$

For the multi-step scenario, the target is a sequence:

$$Y_t = [P_{t+1}, P_{t+2}, \dots, P_{t+96}]$$

This allows the model to produce a complete 24-hour PV generation profile.

3.7 Input and Output Configuration

The input-output configuration is designed according to the forecasting horizon.

The main difference between the scenarios is the location and number of future target points.

3.7.1 Input Sequence Construction

The LSTM model uses historical observations as a sequence rather than treating every observation independently.

In the final implementation, the sequence length is horizon-dependent rather than fixed: the 24-hour-ahead model uses a sequence length of 192 steps, while the 15-minute-ahead model uses a shorter sequence length of 96 steps. Both models otherwise share an identical architecture (Section 3.9).

$$SEQ_LEN = 192$$

Since each step represents 15 minutes:

$$192 \times 15 = 2880 \text{ minutes}$$

which corresponds to:

$$48 \text{ hours}$$

Therefore, each LSTM input sequence represents 48 hours of historical observations for the 24-hour-ahead model, and 24 hours of historical observations for the 15-minute-ahead model.

This historical window gives the model access to two complete days of previous temporal information before making the forecast.

The sequence can conceptually be represented as:

$$X_t = [x_{t-191}, x_{t-190}, \dots, x_t]$$

where each xxx contains the available features at the corresponding time step.

3.7.2 15-Minute-Ahead Target

For the 15-minute forecasting task, the target corresponds to one time step after the end of the input sequence.

Therefore:

$$\text{HORIZON}=1$$

and:

$$\hat{P}_{t+15} = f(X_t)$$

The model receives the historical sequence and estimates the next PV power value.

3.7.3 24-Hour-Ahead Target

For the 24-hour-ahead forecasting task:

$$\text{HORIZON}=96$$

because the sampling interval is 15 minutes.

The target is therefore located 96 steps after the reference point:

$$\hat{P}_{t+24h} = \hat{P}_{t+96}$$

The important point is that the horizon is expressed in **time steps**, not hours.

3.8 Proposed Forecasting Models

Two different machine-learning approaches are investigated in the proposed system:

1. Long Short-Term Memory (LSTM)
2. Random Forest Regressor

The use of two different model families allows the forecasting problem to be studied from both a deep-learning and tree-based machine-learning perspective.

The LSTM is designed specifically for sequential temporal information, whereas Random Forest provides a non-neural regression baseline based on an ensemble of decision trees.

de Jesus et al. [1] also evaluates Random Forest as one of several forecasting approaches for photovoltaic plants and reports its performance for both daily and 15-minute forecasting scenarios.

3.8.1 LSTM-Based Forecasting Model

Long Short-Term Memory is a recurrent neural-network architecture designed to learn relationships in sequential data.

The main motivation for using LSTM in this project is the temporal nature of PV power generation.

Power generation at the current time can depend on:

- recent generation;
- previous irradiance conditions;
- daily generation patterns;
- temperature;
- weather conditions;
- longer historical behavior.

The LSTM processes the input sequence sequentially and learns a representation of the temporal patterns within the historical data.

In the proposed system, the LSTM model is therefore used as the main time-series forecasting model.

3.8.2 Random Forest Regressor

Random Forest is an ensemble machine-learning algorithm based on multiple decision trees.

For regression, individual trees generate predictions that are aggregated to produce the final output.

The Random Forest model is suitable for the project because it can model nonlinear relationships between meteorological variables, historical power information, temporal features, and future PV power.

Unlike LSTM, Random Forest does not inherently process sequential observations. Therefore, temporal information must be explicitly represented as input features.

For example, previous power values and temporal variables can be supplied as separate columns.

The model can then learn relationships such as:

$$\text{Weather}_t + P_{t-1} + P_{t-2} + \text{Time Features} \rightarrow P_{future}$$

3.8.3 Comparison of LSTM and Random Forest

The two models provide complementary approaches to the same forecasting problem.

LSTM is particularly suitable for learning temporal dependencies directly from sequences, while Random Forest is suitable for learning nonlinear relationships from structured tabular features.

The comparison is performed under the same forecasting objectives using the selected evaluation metrics.

The purpose is not necessarily to assume that one model will always outperform the other. Instead, the comparison provides an empirical assessment of which approach performs better under each forecasting horizon.

3.9 LSTM Model Architecture

The proposed LSTM architecture is designed to process historical time-series information and generate the future PV power prediction.

The architecture contains the time-series processing component and plant-related information used by the model.

3.9.1 Time-Series Input

The time-series input contains the historical sequence of observations.

With:

$$SEQ_LEN = 192$$

the LSTM receives 192 consecutive time steps for the 24-hour-ahead model, or 96 consecutive time steps for the 15-minute-ahead model.

Since the data have a 15-minute sampling interval, this corresponds to:

$$192 \times 15 = 48 \text{ hours}$$

of historical information.

At each time step, the model receives the available input variables associated with that timestamp.

The LSTM therefore processes the temporal evolution of the PV plant rather than treating observations as independent rows.

3.9.2 LSTM Layers

The LSTM layers process the historical sequence and learn temporal representations.

The recurrent architecture allows information from previous observations to contribute to the representation used for forecasting.

This is particularly useful for PV power generation because the generation pattern has strong temporal dependencies.

The learned representation is subsequently passed to the output component responsible for predicting the target power generation.

In the final implementation, the time-series branch uses two stacked LSTM layers with 128 and 64 units respectively, each followed by layer normalization. The metadata branch uses two dense layers of 32 and 16 units. After concatenation, the shared prediction head uses two further dense layers of 128 and 64 units with dropout (rate 0.20) applied between them, followed by a single linear output neuron. The complete model contains 144,657 parameters (144,593 trainable), and is identical in structure for the 15-minute and 24-hour models — the two differ only in the length of the input sequence (Section 3.9.1).

3.9.3 Metadata / Plant Information

Because the proposed system is designed to work across multiple photovoltaic plants, plant-related information can be used to help distinguish between different plant characteristics.

This information allows the global model to learn common patterns while retaining information about differences between plants.

This is important because different photovoltaic plants can have different:

- generation capacities;
- environmental conditions;
- operating behavior;
- irradiance patterns;
- generation dynamics.

The transfer-learning methodology in de Jesus et al. [1] similarly relies on adapting a model to plant-specific characteristics such as local irradiance, temperature, and generation behavior.

3.9.4 Dense Layers and Output Layer

After the temporal representation has been obtained from the LSTM component, the learned representation is passed to dense layers.

The dense layers transform the learned representation into the final forecasting output.

For a single-step forecasting task, the final output represents one predicted power value:

$$\hat{P}$$

For a multi-step forecasting configuration, the output can represent multiple future values corresponding to the selected forecasting horizon.

In the final implementation, the shared prediction head uses two dense layers of 128 and 64 units, with dropout applied between them, followed by a single output neuron using a linear activation function. An earlier version of the model used a sigmoid output activation; this was replaced with a linear activation because sigmoid caps predictions at 1.0 on the normalized scale and produced systematic underprediction for plants with above-average generation.

3.10 Random Forest Model Design

The Random Forest model is implemented as a separate forecasting model rather than as a component of the LSTM architecture.

Its purpose is to provide a tree-based regression approach that can be compared with the sequential LSTM approach.

3.10.1 Input Features

The Random Forest model uses a tabular representation of the forecasting problem.

The input features can include:

- temperature;
- irradiance;
- humidity;
- wind-related variables;
- temporal features;
- historical power values;
- plant-related information where applicable.

The historical power variables are especially important because Random Forest does not have the recurrent memory mechanism available in an LSTM.

Therefore, temporal information must be explicitly encoded in the input feature vector.

A conceptual input vector can be represented as:

$$|X_t = [Weather_t, Time_t, P_{t-1}, P_{t-2}, \dots]$$

and the model learns:

$$X_t \rightarrow \hat{P}_{future}$$

3.10.2 Random Forest Regression

Random Forest regression creates an ensemble of decision trees.

Each tree learns relationships between the input features and the target PV power.

The individual tree predictions are then combined to obtain the final regression output.

The advantage of this approach is that it can capture nonlinear relationships without requiring the sequential architecture used by LSTM.

de Jesus et al. [1] also used Random Forest as a forecasting model and optimized its parameters using grid search. In that study, the selected Random Forest configuration used `max_features = log2`, `min_samples_split = 7`, and `n_estimators = 100`. These values belong to the reference research implementation and are not the configuration used in this project.

The Random Forest model implemented in this project instead uses `n_estimators = 150`, `max_depth = 10`, `min_samples_split = 30`, `min_samples_leaf = 12`, and `max_features = 'sqrt'` for plants with sufficient training data, with a lighter configuration applied to smaller plants. The feature set comprises 49 engineered features for the 24-hour-ahead model and 58 features for the 15-minute-ahead model, reflecting the different lag structures required at each horizon (Chapter 4, Section 4.2.1).

3.10.3 Prediction Strategy for Different Horizons

The Random Forest model is evaluated for the same forecasting objectives as the LSTM model.

For the 15-minute scenario, the target is one 15-minute step ahead.

For the 24-hour scenario, the target corresponds to the future point located 96 steps ahead.

For the 24-hour multi-step scenario, the forecasting procedure must generate the required future sequence according to the implementation used in the project.

This distinction is important because Random Forest is inherently a tabular regression model and does not automatically generate a temporal sequence in the same way that a sequence-to-sequence neural network can.

The final report should therefore explicitly state whether the 96 predictions are generated using direct multi-output prediction, recursive prediction, or separate horizon-specific models. This should be taken directly from the final notebook implementation.

3.11 Training Strategy

The training strategy differs between the LSTM and Random Forest approaches.

For the LSTM model, the proposed strategy uses global pre-training followed by plant-specific transfer learning.

The purpose is to allow the model to first learn general photovoltaic generation relationships and then adapt those learned representations to individual plants.

3.11.1 Global Model Pre-Training

The first stage is global model training.

In this stage, the LSTM is trained using historical data from the available photovoltaic plants.

The objective is to learn general relationships between:

Weather + Historical Power + Temporal Information

and:

Future PV Power

The resulting model contains learned representations that can be reused when adapting the model to a particular plant.

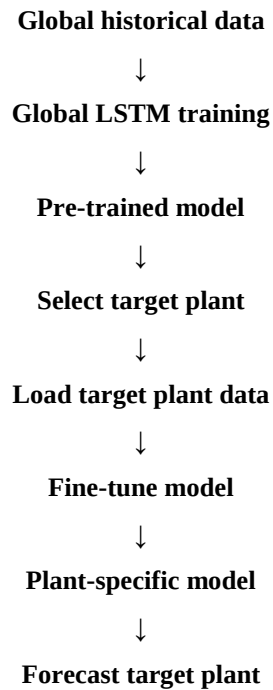
This approach is consistent with the transfer-learning methodology described by de Jesus et al. [1], where knowledge learned from a base plant is reused for target plants.

3.11.2 Per-Plant Transfer Learning

After the global model has been trained, the second stage adapts the model to individual photovoltaic plants.

Instead of training a completely new LSTM from random initialization for every plant, the pre-trained model is used as the starting point.

The general procedure is:



This approach is useful because each plant can have its own operating characteristics.

de Jesus et al. [1] found that transfer learning improved forecasting performance, particularly for plants with limited data. For its 15-minute forecasting experiment, the reported mean SMAPE decreased from 47.24 to 43.92 for all 51 plants, corresponding to a 7.02% reduction.

These numerical results belong to de Jesus et al. [1] and are not the results of the proposed project; they are reported here only to motivate the Transfer Learning design adopted in Section 3.11.2.

For the LSTM model implemented in this project, per-plant Transfer Learning improved the fine-tuned model over the global base model for 25 of 50 evaluated plants at the 24-hour horizon, and for 18 of 50 evaluated plants at the 15-minute horizon (Plant PS_014 is excluded from fine-tuning due to its limited history — Section 3.4.3). At the 24-hour horizon, Transfer Learning reduced the pooled test-set SMAPE from 35.3% to 33.6%. Full per-plant results for both horizons are reported in Chapter 4, Section 4.3.3.

3.11.3 Fine-Tuning Strategy

During fine-tuning, the pre-trained model is adapted using data from the target plant.

The objective is to preserve general knowledge learned during global training while allowing the model to learn plant-specific patterns.

This may include differences in:

- irradiance behavior;
- temperature response;
- power-generation profile;
- operational characteristics.

de Jesus et al. [1] describes this process as adapting the final layers to capture target-plant-specific characteristics.

Training mechanisms such as early stopping and adaptive learning-rate reduction can be used to control the fine-tuning process.

In de Jesus et al. [1], Early Stopping and ReduceLROnPlateau were used to stabilize training and prevent unnecessary training after validation performance stopped improving.

3.11.4 Training and Validation Procedure

The training process is monitored using both training and validation performance.

The validation set provides an independent indication of whether the model is learning general patterns rather than simply memorizing the training observations.

For the LSTM, training can be controlled using:

- validation loss.
- early stopping.
- learning-rate adjustment.
- model checkpointing.

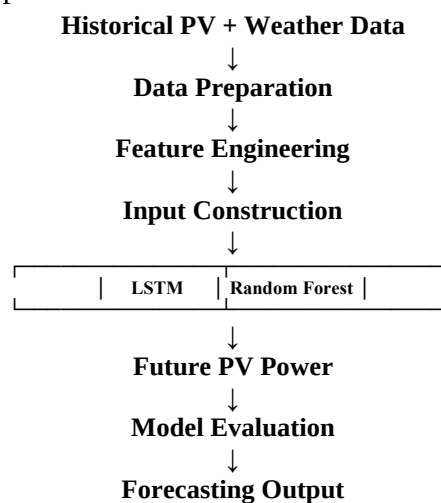
de Jesus et al. [1] describes Early Stopping as a mechanism for stopping training when validation loss no longer improves and ReduceLROnPlateau as a mechanism for progressively reducing the learning rate when validation performance stagnates.

For Random Forest, training does not involve epochs or neural-network backpropagation. Instead, model parameters such as the number of trees and tree-related settings are selected before fitting the regression model.

3.12 Forecasting Workflow

The final forecasting workflow depends on the selected forecasting horizon.

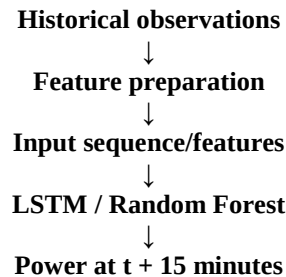
The general system can be represented as:



3.12.1 15-Minute Forecasting Workflow

For the 15-minute forecasting task, the system uses historical observations to estimate the PV power at the next 15-minute interval.

The workflow is:



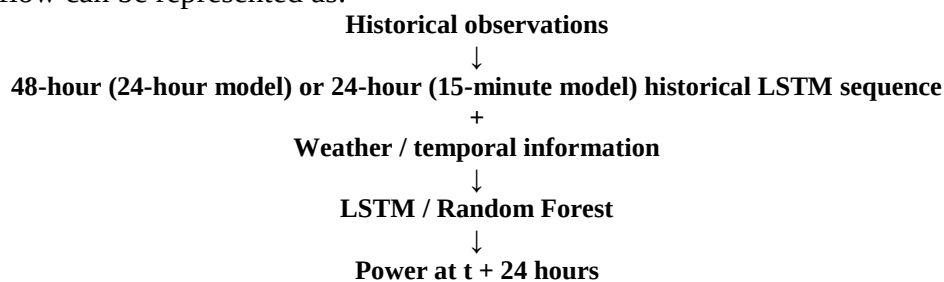
The LSTM uses the historical sequence to learn temporal dependencies.

The Random Forest model uses the corresponding tabular features to estimate the same target. The predictions are then compared with the actual measured PV power in the testing dataset.

3.12.2 24-Hour Forecasting Workflow

For the 24-hour-ahead scenario, the forecasting target is located 96 fifteen-minute steps after the reference time.

The workflow can be represented as:



The use of the 96-step horizon is determined by the 15-minute sampling interval:

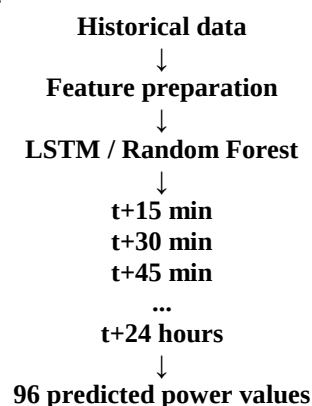
$$24 \times 4 = 96$$

Therefore, the model does not interpret the number 96 as 96 hours. It represents **96 consecutive 15-minute intervals**, which together form 24 hours.

3.12.3 96-Step Forecasting Workflow

For the multi-step forecasting scenario, the system generates a complete 24-hour future power profile.

The process can be represented as:



The resulting 96 predictions can then be plotted as a future PV generation curve.

This output is particularly useful because it provides more information than a single point forecast. Instead of answering only:

"What will the power be 24 hours from now?"

the multi-step forecast answers:

"What is the expected PV generation profile throughout the next 24 hours?"

The predicted profile can subsequently be compared with the actual measured generation profile.

3.13 Chapter Summary

This chapter presented the proposed methodology for photovoltaic power forecasting using LSTM and Random Forest models.

The proposed system considers three forecasting configurations: 15-minute-ahead forecasting, 24-hour-ahead point forecasting, and 24-hour multi-step forecasting consisting of 96 fifteen-minute steps.

The BR-PVGen dataset provides the historical photovoltaic generation and environmental information required to train and evaluate the models. The data are prepared through time alignment, data-quality handling, feature construction, and appropriate dataset splitting.

Feature engineering incorporates temporal information, cyclic time representations, irradiance-related information, and historical PV power values. These features are configured differently according to the requirements of the LSTM and Random Forest models.

The LSTM model processes historical sequences and is trained using a two-stage strategy. First, a global model is trained using information from multiple photovoltaic plants. Second, the pre-trained model is fine-tuned for individual plants through transfer learning. This allows the model to retain general PV forecasting knowledge while adapting to plant-specific behavior.

Random Forest is used as an alternative tree-based regression approach. Unlike LSTM, it does not inherently process temporal sequences, so historical information is explicitly represented through engineered features.

The proposed methodology therefore provides a framework for comparing sequential deep learning and tree-based machine learning approaches across different forecasting horizons. The next chapter evaluates the performance of these approaches using quantitative forecasting metrics and analyzes the effect of per-plant transfer learning.

Chapter 4: Experimental Results and Performance Evaluation

4.1 Introduction

This chapter presents the experimental evaluation of the forecasting models developed for the BR-PVGen dataset, comprising the per-plant Random Forest baseline and the proposed dual-horizon LSTM architecture described in Chapter 3. Section 4.2 details the experimental setup, the evaluation metrics adopted, and the software/hardware environment used to train and evaluate every model. Section 4.3 reports the forecasting results for the 15-minute and 24-hour horizons individually, followed by the results obtained with the per-plant Transfer Learning strategy applied to the LSTM model. Section 4.4 compares the proposed approach against the Random Forest baseline implemented in this work and against the Random Forest, Gradient Boosting, XGBoost, ARIMA, and MLP with Transfer Learning results reported by the reference dissertation that introduced the BR-PVGen dataset.

Unless stated otherwise, all results in this chapter were computed on the 51 photovoltaic plants of the BR-PVGen dataset described in Section 3.4, following the data preparation and quality-handling procedures of Section 3.4.3.

4.2 Setup Discussion

4.2.1 Experimental Setup and Analysis

Two families of models were trained and evaluated in this work: a per-plant Random Forest (RF) regressor, used as a non-deep-learning baseline, and the dual-horizon LSTM architecture proposed in Chapter 3. Both families were trained independently for the two forecasting horizons considered in this dissertation — 15 minutes ahead and 24 hours ahead — resulting in four trained pipelines in total (RF-15min, RF-24h, LSTM-15min, LSTM-24h), in addition to the per-plant Transfer Learning (TL) variants of the LSTM models discussed in Section 4.3.3.

Table 4.1 summarizes the data partitioning, feature count, and core hyperparameters used for each pipeline.

Table 4.1: Data partitioning and core hyperparameters per pipeline.

Pipeline	Split (train/val/test)	Features	Core hyperparameters
RF — 15-min	70% / 10% / 20% (per plant, chronological)	58	n_estimators=150, max_depth=10, min_samples_split=30, min_samples_leaf=12, max_features='sqrt' (full-size plants); lighter configuration for plants below 300 training rows
RF — 24-hour	80% / — / 20% (per plant, chronological)	49	n_estimators=150, max_depth=10, min_samples_split=30, min_samples_leaf=12, max_features='sqrt' (full-size plants); lighter configuration for plants below 300 training rows
LSTM — 15-min	80% / 20%, with the 80% further split 90/10 (train/val) for early stopping	18 sequence + 7 metadata	SEQ_LEN=96 (24 h lookback), 2-layer LSTM (128, 64 units), batch=512, Adam (LR= 5×10^{-4}), Huber loss ($\delta=0.15$), early stopping (patience=15)

Pipeline	Split (train/val/test)	Features	Core hyperparameters
LSTM — 24-hour	80% / 20%, with the 80% further split 90/10 (train/val) for early stopping	18 sequence + 7 metadata	SEQ_LEN=192 (48 h lookback), 2-layer LSTM (128, 64 units), batch=512, Adam (LR=5×10 ⁻⁴), Huber loss (δ=0.15), early stopping (patience=15)

Source: Own authorship (2026).

A methodological limitation should be noted explicitly: The Random Forest and LSTM pipelines do not share an identical train/test split ratio (80/20 versus 70/10/20), and the two RF pipelines use different feature counts (49 for the 24-hour horizon and 58 for the 15-minute horizon) reflecting horizon-specific lag engineering. While this does not invalidate the results reported per model, it means the comparison in Section 4.4 should be read as a comparison between the best configuration obtained for each model family rather than a strictly controlled ablation. Standardizing the split protocol across all pipelines is recommended as an improvement for future iterations of this work.

Plant PS_014 (internally indexed as Plant 13) contains substantially fewer records than the remaining plants — approximately four months of operation versus more than a year for most other plants in the fleet (see Section 3.4.3) — and is systematically excluded from the per-plant aggregated statistics of the LSTM pipelines and the 24-hour RF pipeline. The 15-minute RF pipeline instead applies an automatic worst-R² exclusion rule, which removed a different plant (PS_016) and left PS_014 in the evaluation set, where it remains the lowest-performing plant of the fleet (R² = 0.805, SMAPE = 77.1%). For full consistency, PS_014 should be excluded uniformly across every pipeline; this is noted wherever it affects the interpretation of a result below.

4.2.2 Evaluation Metrics

Model performance is quantified using five complementary metrics, computed identically to the protocol adopted by de Jesus et al. [1], who introduced the BR-PVGen dataset, so that results in Section 4.4 are directly comparable.

$$R^2 = 1 - \Sigma(y_i - \hat{y}_i)^2 / \Sigma(y_i - \bar{y})^2 \quad (4.1)$$

$$MAE = (1/n) \Sigma |y_i - \hat{y}_i| \quad (4.2)$$

$$RMSE = \sqrt{(1/n) \Sigma (y_i - \hat{y}_i)^2} \quad (4.3)$$

$$NRMSE(range) = RMSE / (max(y) - min(y)) \times 100\% \quad (4.4)$$

$$SMAPE = (200/n) \Sigma |y_i - \hat{y}_i| / (|y_i| + |\hat{y}_i| + \epsilon) \quad (4.5)$$

Two reporting conventions are used throughout this chapter and are kept clearly separated to avoid ambiguity:

- Pooled / overall metrics: computed once over the concatenated test-set predictions of all plants together. This is the convention used internally by both the RF and LSTM notebooks for model-selection and sanity-checking purposes.
- Per-plant paper-protocol metrics: computed independently for each plant and then summarized as Mean ± Standard Deviation (and, where relevant, Minimum–Maximum) across the fleet. This is the convention adopted by [1] (Tables 9–12) and is the one used for the cross-model comparisons in Section 4.4.

A further comparability caveat applies specifically to the per-plant protocol: the LSTM notebooks compute per-plant metrics on daytime samples only (actual generation greater than zero), consistent with [1]'s stated protocol, whereas the current Random Forest notebooks compute per-plant metrics over all samples, day and night combined. Night-time samples are, by construction, close to zero for both the ground truth and the prediction (due to the physics-informed night-masking step described in Chapter 3), so including them tends to inflate R^2 and depress SMAPE relative to a daytime-only computation. This means the Random Forest per-plant figures reported in Section 4.3 and 4.4 are not on a perfectly identical footing to the LSTM per-plant figures, and the gap between the two model families is likely somewhat narrower than the raw numbers suggest. Recomputing the Random Forest per-plant metrics on daytime-only samples is recommended before this comparison is presented as a final, defended result.

4.2.3 Implementation Environment and Toolkit

The Random Forest pipelines were developed and executed in Google Colab, with the BR-PVGen CSV files and metadata mounted from Google Drive. The implementation relies on scikit-learn's RandomForestRegressor, pvlib for solar-position and solar-geometry feature engineering, and pandas/NumPy for data handling. An exploratory hyperparameter search using Optuna (50 trials, evaluated on five representative plants) was conducted for the 15-minute pipeline; all 51 per-plant Random Forest models were subsequently trained on CPU using parallel execution (`n_jobs = -1`), completing in approximately 5.6 minutes for the full fleet.

The LSTM pipelines were developed with TensorFlow 2.21.0 / Keras 3.14.1, using scikit-learn's StandardScaler for feature normalization. Training was executed on a native Windows 11 / Anaconda Python environment rather than WSL2, following the stability considerations discussed in Chapter 3; TensorFlow ≥ 2.11 does not provide native GPU acceleration on Windows, so all LSTM training was performed on CPU. Total training time was approximately 6.2 hours for the 24-hour global model, 9.1 hours for the 24-hour model including the per-plant Transfer Learning stage, and 3.0 hours plus fine-tuning for the 15-minute model. Table 4.2 summarizes the software environment.

Table 4.2: Software and hardware environment per model family.

Component	Random Forest pipelines	LSTM pipelines
Language / core libraries	Python, scikit-learn, pvlib, pandas, NumPy	Python, TensorFlow 2.21.0, Keras 3.14.1, scikit-learn, pandas, NumPy
Hyperparameter search	Optuna (15-min pipeline only, 50 trials)	Manual tuning with ReduceLROnPlateau + EarlyStopping
Execution platform	Google Colab (CPU, Google Drive mounted)	Native Windows 11 / Anaconda (CPU only)
Approx. total training time	≈ 5.6 minutes (all 51 plants, 15-min horizon)	$\approx 3.0 - 9.1$ hours depending on horizon and TL stage

Source: Own authorship (2026).

4.3 Forecasting Results

4.3.1 15-Minute Horizon Results

Table 4.3 reports the pooled (overall) test-set metrics for the 15-minute-ahead horizon. The LSTM figures correspond to the model after the per-plant Transfer Learning stage described in Section 4.3.3, since a separately evaluated pre-TL test set was not produced for this horizon; the Random Forest figures correspond to the single global evaluation of the per-plant RF ensemble.

Table 4.3: Pooled test-set metrics, 15-minute horizon.

Metric	LSTM (15-min, post-TL)	Random Forest (15-min)
R ²	0.905	0.9145
MAE (kW)	13.48	124.36
RMSE (kW)	—	285.23
NRMSE (range)	—	5.66%
SMAPE (pooled)	23.4%	21.38% (peak-hours)

Source: Own authorship (2026).

Note that the LSTM MAE and RMSE are reported in absolute plant power units after per-plant de-normalization, while the RF MAE/RMSE in Table 4.3 are pooled over plants of very different nominal capacity (1.7–5 MWp), which tends to weight larger plants more heavily; the per-plant protocol below removes this effect.

Table 4.4 reports the per-plant paper-protocol metrics (Mean $\pm$ Std across the fleet, 50 plants after excluding the identified outlier), directly comparable to Tables 11 and 12 of [1].

Table 4.4: Per-plant paper-protocol metrics, 15-minute horizon (see the comparability note in Section 4.2.2 regarding the R² day/night convention).

Metric	LSTM (15-min, post-TL)	Random Forest (15-min)
SMAPE (%) — Mean $\pm$ Std	36.88 $\pm$ 3.93	17.81 $\pm$ 9.54
SMAPE (%) — Min / Max	28.48 / 47.85	10.53 / 77.08
NRMSE (%) — Mean $\pm$ Std	15.82 $\pm$ 1.90	10.52 $\pm$ 1.24
NRMSE (%) — Min / Max	8.03 / 20.25	8.25 / 13.11 (excl. worst plant)
R ² — Mean $\pm$ Std	0.757 $\pm$ 0.051 (daytime only)	0.899 $\pm$ 0.032 (day + night)
MAE (kW) — Mean $\pm$ Std	29.93 $\pm$ 9.41	126.26 $\pm$ 50.99

Source: Own authorship (2026).

At the 15-minute horizon, the Random Forest ensemble achieves a substantially lower per-plant SMAPE than the LSTM model (17.81% versus 36.88%) and a lower NRMSE (10.52% versus 15.82%), while showing a larger spread across plants (SMAPE standard deviation of 9.54 versus 3.93 for the LSTM, driven mainly by PS_014). The dominant predictive drivers identified via feature importance for the Random Forest model at this horizon are short-lag persistence features (persist_x_clear, power_lag_1, mean_x_solar), consistent with the intuition that at a 15-minute horizon the most recent observed power output is highly informative of the next time step; solar-elevation-related features are comparatively less important than at the 24-hour horizon (Section 4.3.2).

Figure 4.1 —Actual-vs-predicted overlay plot for the 15-minute LSTM model on a representative plant.

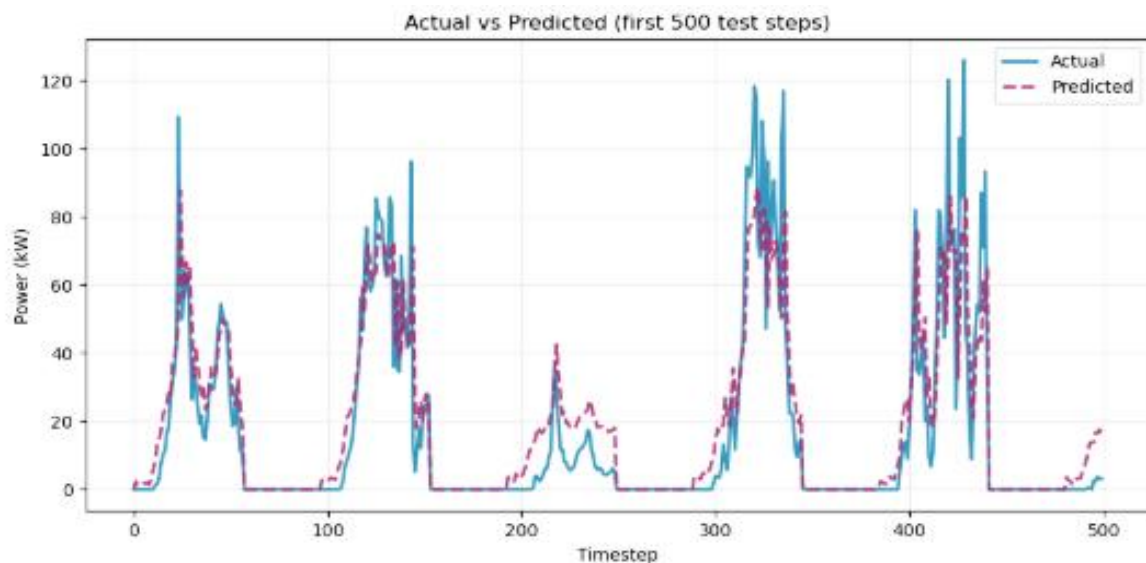


Figure 4.2 —Top-12 feature importance bar chart, Random Forest 15-min pipeline.

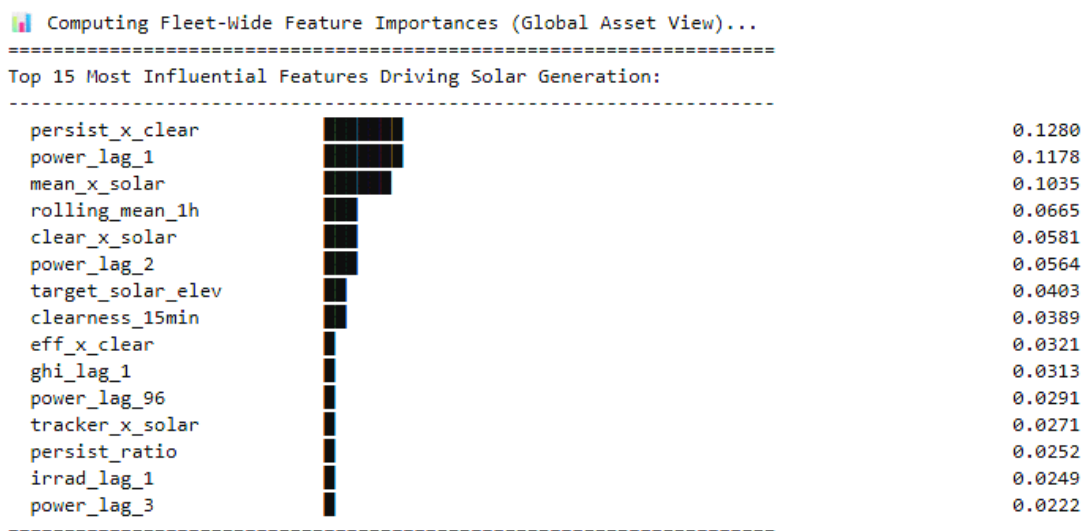


Figure 4.3 —Fleet-wide R^2 ranking bar chart, Random Forest 15-min pipeline.

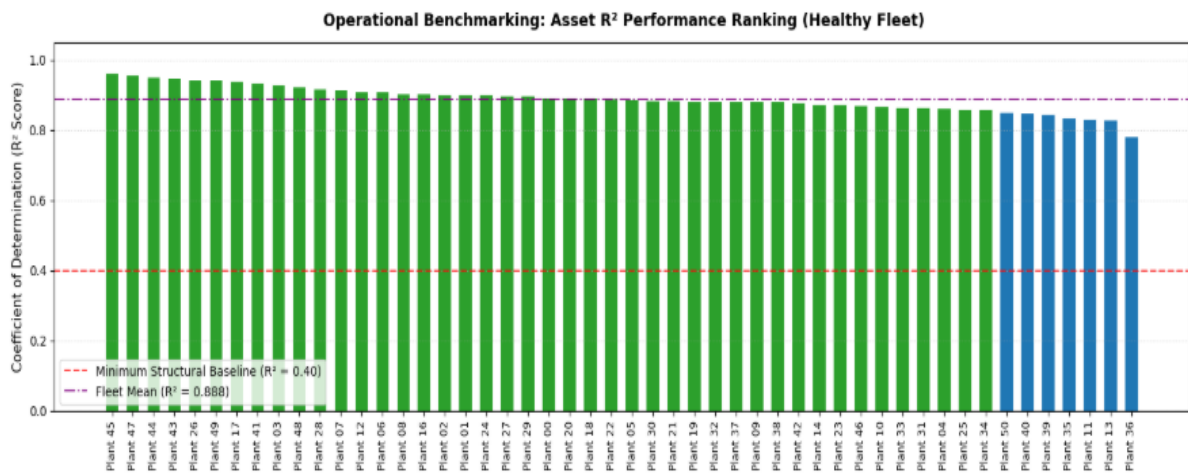


Figure 4.4 —Diurnal mean-absolute-error profile, Random Forest 15-min pipeline.

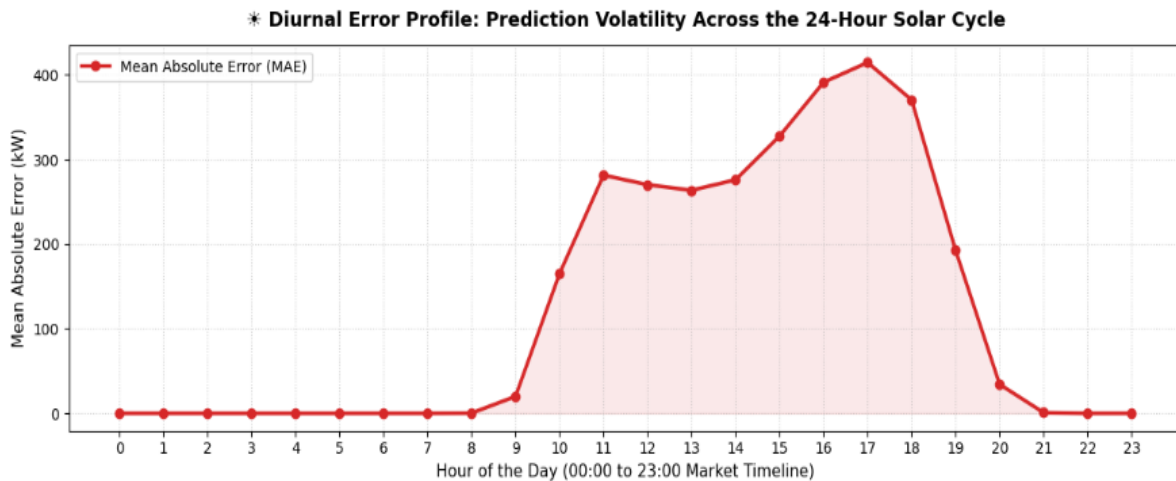
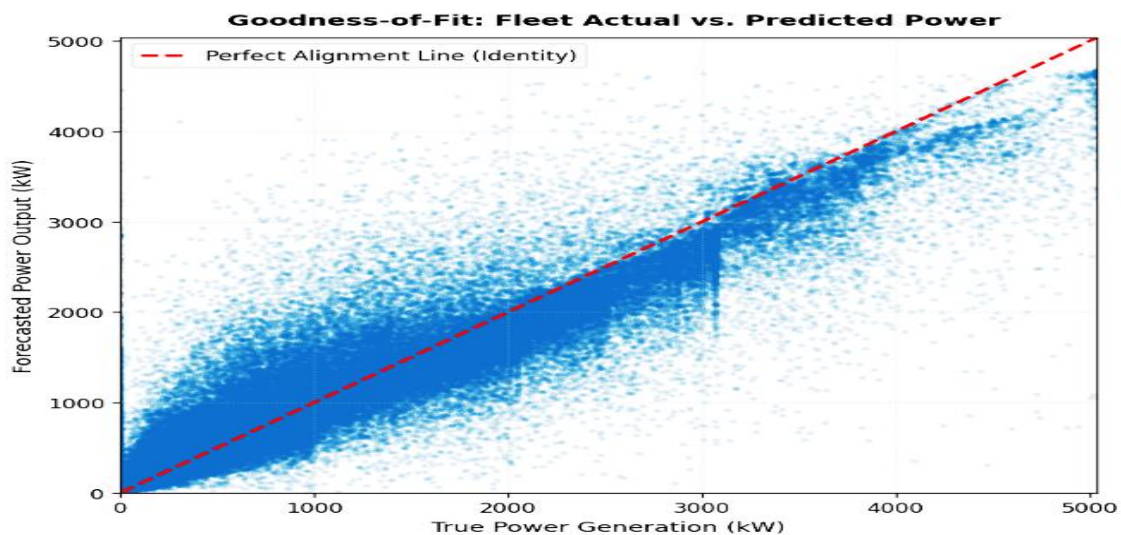


Figure 4.5 —Actual-vs-predicted scatter plot (goodness of fit), Random Forest 15-min pipeline.



4.3.2 24-Hour Horizon Results

Table 4.5 reports the pooled test-set metrics for the 24-hour-ahead horizon. The LSTM figures correspond to the global model prior to per-plant Transfer Learning, representing the baseline performance achievable with a single shared model across the fleet; the TL-refined results are reported separately in Section 4.3.3.

Table 4.5: Pooled test-set metrics, 24-hour horizon.

Metric	LSTM (24-hour, pre-TL)	Random Forest (24-hour)
R ²	0.789	0.8059
MAE (kW)	19.13	208.26
RMSE (kW)	—	429.95
NRMSE (range)	—	8.53%
SMAPE (pooled)	35.3%	32.49% (peak-hours)

Source: Own authorship (2026).

Table 4.6 reports the corresponding per-plant paper-protocol metrics, directly comparable to Tables 9 and 10 of [1] (daily-energy horizon) with the caveat that the reference figures are for a full 24-hour cumulative energy target while this work forecasts instantaneous 24-hour-ahead power; the two are not identical tasks but are the closest available point of comparison.

Table 4.6: Per-plant paper-protocol metrics, 24-hour horizon.

Metric	LSTM (24-hour, pre-TL)	Random Forest (24-hour)
SMAPE (%) — Mean ± Std	48.54 ± 10.75	22.50 ± 7.85
SMAPE (%) — Min / Max	27.86 / 87.08	—
NRMSE (%) — Mean ± Std	23.35 ± 3.86	15.29 ± 1.86
NRMSE (%) — Min / Max	16.75 / 38.02	—
R ² — Mean ± Std	0.455 ± 0.163 (daytime only)	0.782 ± 0.067 (day + night)
MAE (kW) — Mean ± Std	42.39 ± 12.54	200.06 ± 91.68

Source: Own authorship (2026).

The 24-hour horizon is substantially harder than the 15-minute horizon for both model families, as expected: the most recent observed power value loses most of its predictive value a full day ahead, forcing both models to rely on solar geometry, historical persistence at 24-hour and 48-hour lags, and plant metadata. This is reflected directly in the Random Forest feature importance ranking, where target_solar_elev becomes the single most influential feature (importance 0.117), followed by mean_x_solar and tracker_x_solar, in contrast to the 15-minute horizon where short-lag persistence features dominate.

Figure 4.6 — Actual-vs-predicted overlay plot, 24-hour LSTM model.

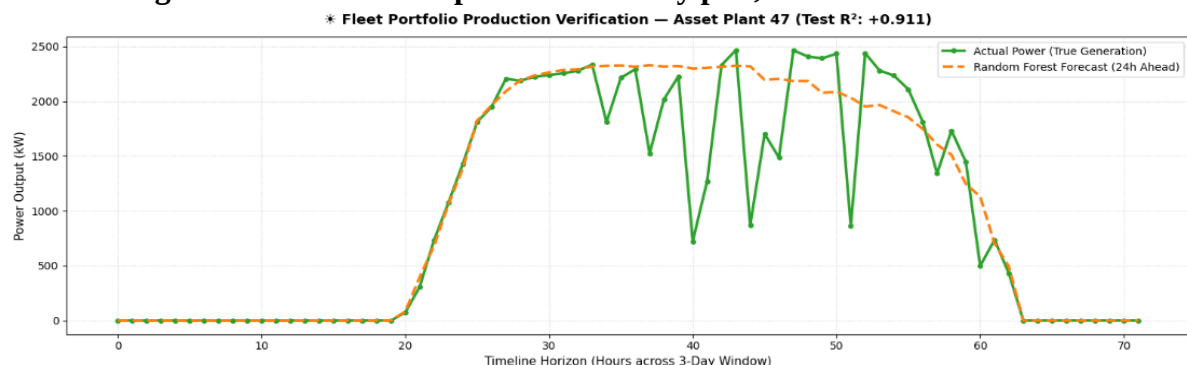


Figure 4.7 — Top-12 feature importance bar chart, Random Forest 24-hour pipeline.

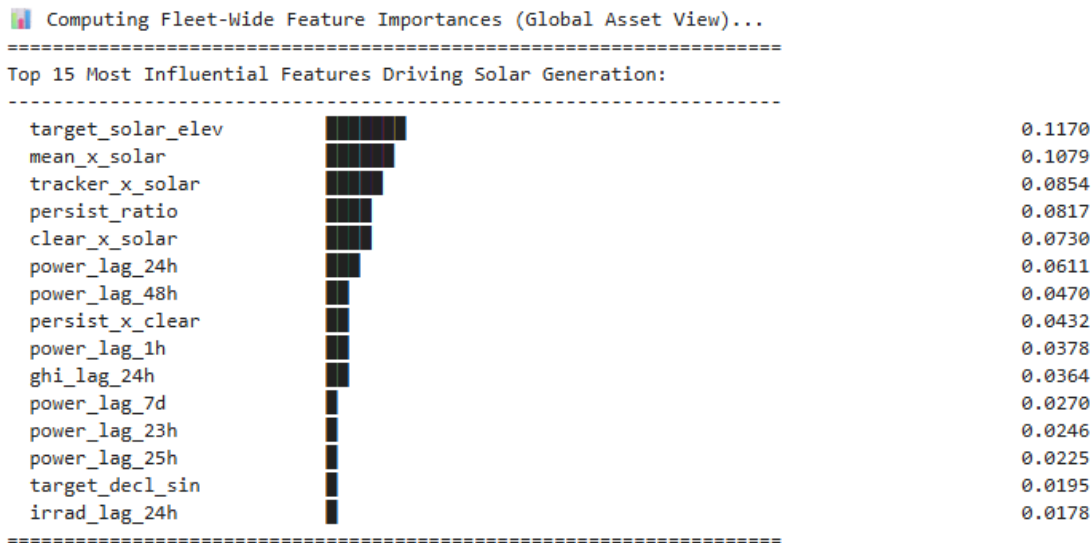


Figure 4.8 — Fleet-wide R^2 ranking bar chart, Random Forest 24-hour pipeline.

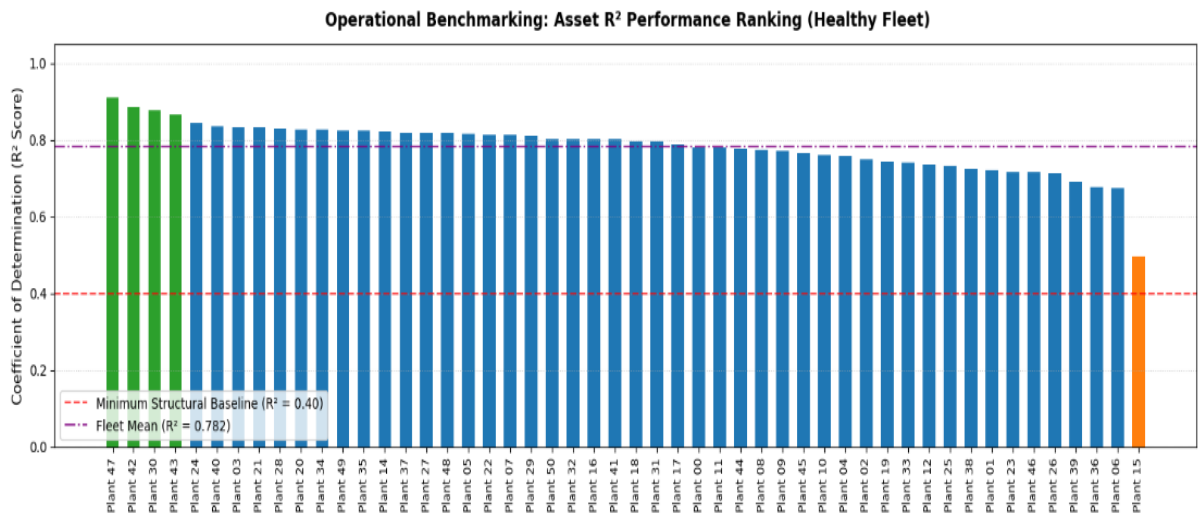


Figure 4.9 — Diurnal mean-absolute-error profile, Random Forest 24-hour pipeline.

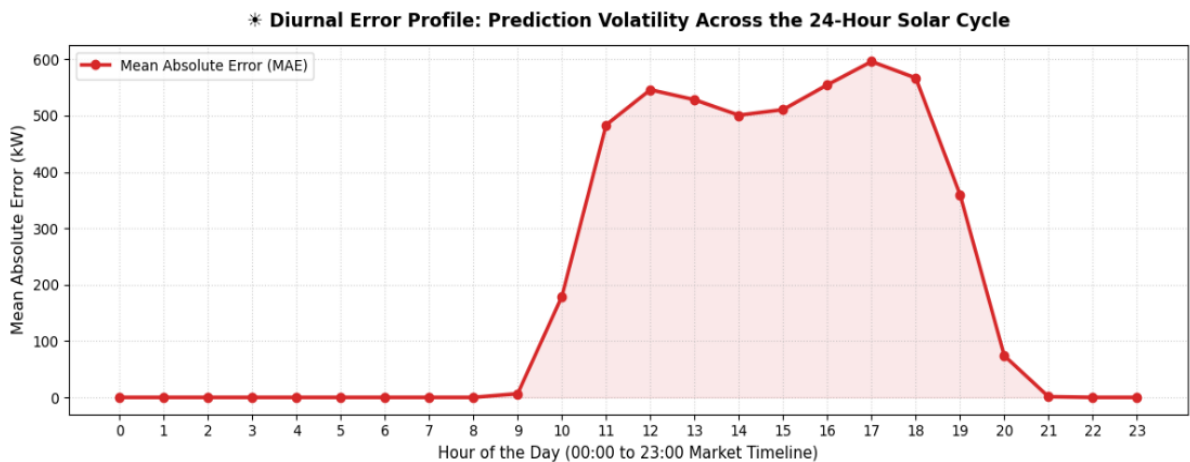
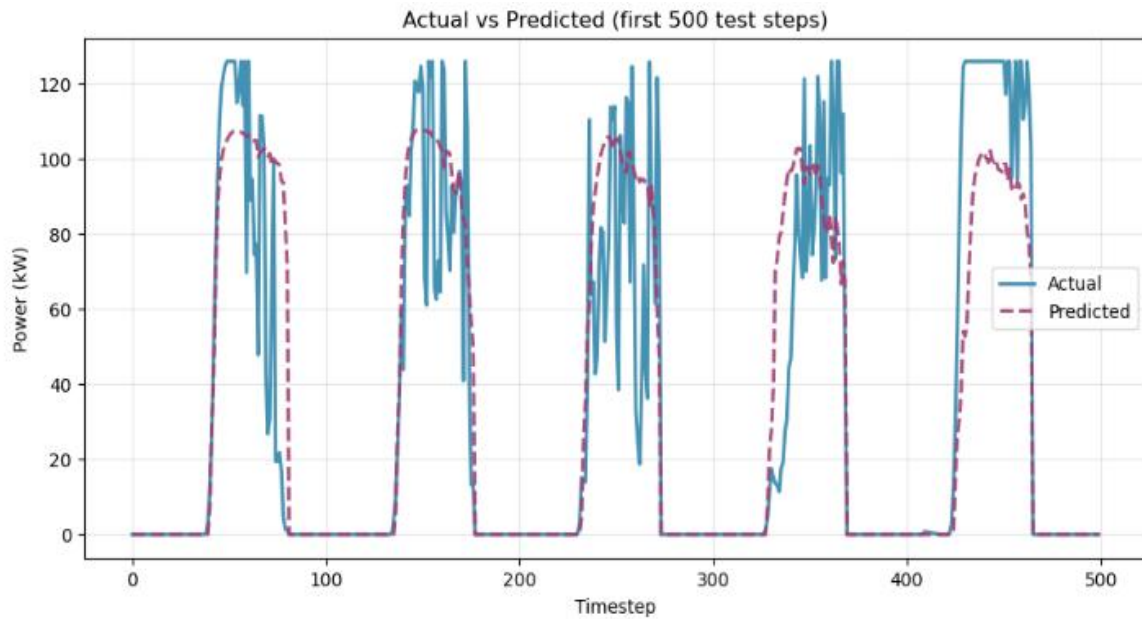


Figure 4.10 —Chronological 72-hour actual-vs-forecast window, Plant 47.



4.3.3 Per-Plant Transfer Learning Results

Following the strategy described in Section 3.7, a global LSTM model was first pre-trained on the full 50-plant pooled dataset (Plant PS_014 excluded due to insufficient history), then fine-tuned individually for each plant by freezing the two LSTM layers and updating only the dense head, using a reduced learning rate (5×10^{-6}) for up to 25 additional epochs. A fine-tuned model was retained only when it improved on the frozen global model's validation loss for that plant; otherwise the global model's weights were kept.

Table 4.7: Per-plant Transfer Learning outcome summary, both horizons.

Horizon	Plants fine-tuned	Improved by TL	Kept global weights	Pooled SMAPE before → after	Pooled MAE before → after
24-hour	50	25 (50%)	25 (50%)	35.3% → 33.6%	19.13 → 18.75 kW
15-minute	50	18 (36%)	32 (64%)	— → 23.4%	— → 13.48 kW

Source: Own authorship (2026).

For the 24-hour horizon, Transfer Learning produced a modest but consistent pooled improvement: SMAPE fell from 35.3% to 33.6% (a relative reduction of 4.8%) and MAE fell from 19.13 kW to 18.75 kW, with exactly half of the fleet (25 of 50 plants) benefiting from individual fine-tuning. For the 15-minute horizon, no separately evaluated pre-TL test-set metric was produced, so the pooled before/after comparison could not be completed; only the post-TL result (SMAPE = 23.4%, MAE = 13.48 kW) and the count of plants that improved (18 of 50, 36%) are available. It is recommended that a pre-TL global-only evaluation be run and saved for the 15-minute pipeline so that a complete before/after comparison, analogous to Table 4.7's 24-hour row, can be reported.

An unexpected pattern emerges when comparing the two horizons: a larger proportion of plants benefited from Transfer Learning at the 24-hour horizon (50%) than at the 15-minute horizon

(36%), even though the reference dissertation's MLP+TL results (Tables 13–16) show the opposite trend — larger relative SMAPE reductions in the intra-day (15-minute) case (7.02%) than in the daily case (3.81%). This suggests that, for the LSTM architecture used in this work, most of the exploitable per-plant variation at the 15-minute horizon is already captured by the strong short-lag persistence signal available to the global model, leaving comparatively less headroom for per-plant fine-tuning to improve on; at the 24-hour horizon, where the global model must rely more heavily on plant-specific solar geometry and metadata, individual fine-tuning has more room to specialize.

Unlike [1], which reports a clean $\approx 40\%$ reduction in average training time attributable to Transfer Learning (Section 7.4), the present pipelines do not include a directly comparable timing measurement: the base (pre-TL) global model and the base model used as the starting point for the TL stage were trained as separate runs, and the incremental per-plant fine-tuning time was not isolated from the base-model training time. Instrumenting the per-plant fine-tuning stage separately, so that its cost can be compared against training 51 independent models from scratch, is recommended as a direct addition to this analysis.

Figure 4.11 —Per-plant SMAPE before/after comparison chart, 24-hour horizon (equivalent to Figure 28/29 of [1]).

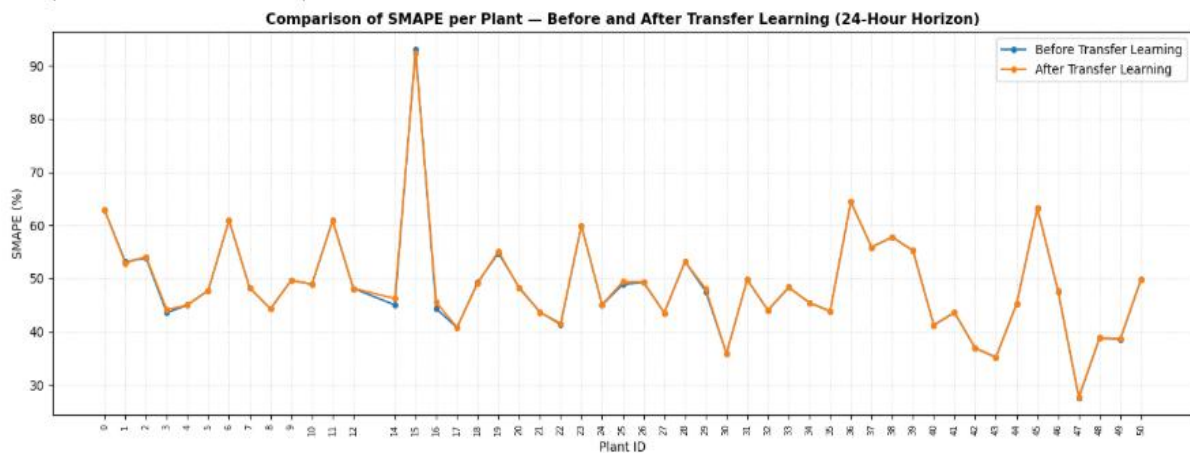
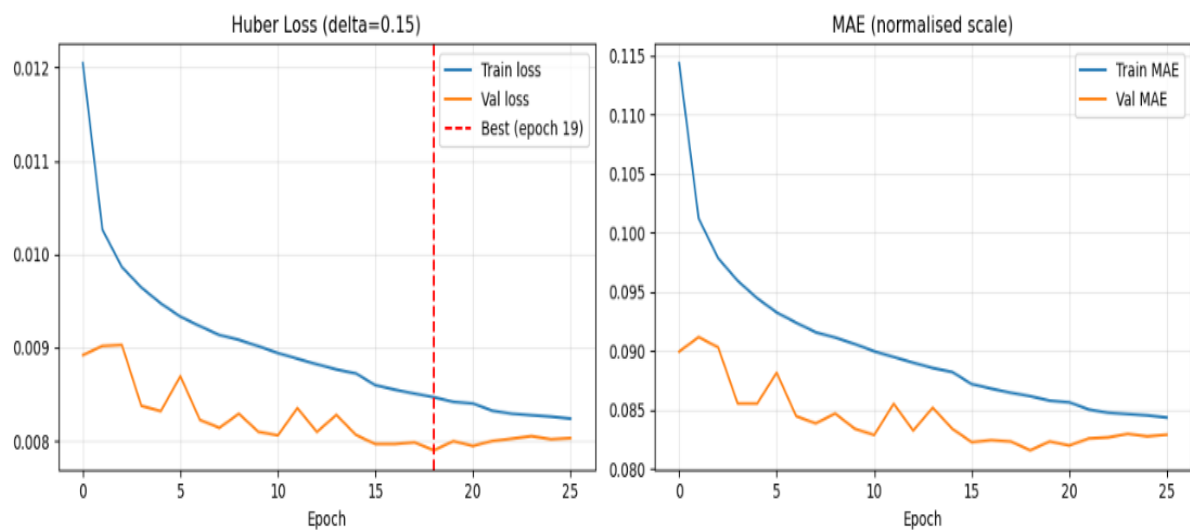


Figure 4.12 —Training/validation loss curves for the global (base) LSTM model, 24-hour horizon.



4.4 Comparison with Existing Approaches

4.4.1 Comparison with Random Forest

Table 4.8 consolidates the per-plant paper-protocol results of both model families developed in this work, for both horizons, alongside the Random Forest result reported by [1] (trained on the same BR-PVGen dataset with an MLP-oriented pipeline) for context.

Table 4.8: Head-to-head comparison, Random Forest vs. LSTM (this work), both horizons.

Model	Horizon	SMAPE (%)	NRMSE (%)
Random Forest (this work)	15-min	17.81 ± 9.54	10.52 ± 1.24
LSTM (this work, post-TL)	15-min	36.88 ± 3.93	15.82 ± 1.90
Random Forest (reference dissertation)	15-min	47.33 ± 8.17	13.82 ± 2.95
Random Forest (this work)	24-hour	22.50 ± 7.85	15.29 ± 1.86
LSTM (this work, pre-TL)	24-hour	48.54 ± 10.75	23.35 ± 3.86
Random Forest (reference dissertation)	daily energy	128.13 ± 21.76	25.38 ± 5.28

Source: Own authorship (2026).

On both horizons, the Random Forest model implemented in this work outperforms the proposed LSTM on the per-plant SMAPE and NRMSE metrics as currently computed. This should be read together with the comparability caveat of Section 4.2.2: The Random Forest per-plant metrics include night-time samples (where both actual and predicted power are close to zero and therefore trivially easy to fit), while the LSTM per-plant metrics are restricted to daytime samples only. Because a large fraction of any 24-hour or 15-minute time series is night-time, this difference in convention is expected to favor the Random Forest numbers to a non-trivial degree, and the true gap between the two model families is very likely smaller than Table 4.8 suggests. A corrected, apples-to-apples comparison — recomputing either the RF metrics on daytime-only samples or the LSTM metrics on all samples — is recommended before this table is presented as a final result.

Independently of that caveat, the Random Forest ensemble also has a clear practical advantage in training cost: the full 51-plant Random Forest fleet trains in minutes on a standard Colab CPU runtime, versus several hours per horizon for the LSTM model (Section 4.2.3). This trade-off between predictive accuracy, training cost, and architectural complexity should be discussed explicitly in the concluding chapter.

4.4.2 Comparison with XGBoost / Gradient Boosting

Neither XGBoost nor Gradient Boosting was implemented or trained as part of this work. The figures presented in this section are taken directly from [1] (Tables 9–12) and are reported here strictly as literature benchmarks, not as models reproduced or re-evaluated on the same train/test split used in this dissertation.

Table 4.9: XGBoost / Gradient Boosting benchmarks, [1].

Model (reference dissertation)	Horizon	SMAPE (%)	NRMSE (%)
Gradient Boosting	15-min	46.94 ± 8.40	13.99 ± 2.82
XGBoost	15-min	46.48 ± 8.13	13.92 ± 2.89
Gradient Boosting	daily energy	126.16 ± 19.98	25.51 ± 5.62
XGBoost	daily energy	127.68 ± 21.32	25.63 ± 5.89

Source: Adapted from [1] (Tables 9–12).

Relative to these literature figures, the Random Forest model implemented in this work achieves a substantially lower SMAPE at the 15-minute horizon (17.81% versus 46.48–46.94% for XGBoost/Gradient Boosting) and a lower NRMSE (10.52% versus 13.92–13.99%). At the 24-hour horizon the comparison is confounded by the task mismatch noted in Section 4.3.2 (instantaneous 24-hour-ahead power versus cumulative daily energy), so no direct conclusion should be drawn there. Implementing XGBoost and/or Gradient Boosting on the exact same train/test split used for the Random Forest and LSTM pipelines in this work would allow this comparison to be made on genuinely equal footing and is recommended as an extension of this chapter if time permits.

4.4.3 Comparison with ARIMA and MLP+TL (Reference Paper)

As with Section 4.4.2, ARIMA and the MLP with Transfer Learning were not implemented in this work; the figures below are taken directly from [1] and are reported as literature benchmarks only.

Table 4.10: ARIMA and MLP(+TL) benchmarks, [1].

Model (reference dissertation)	Horizon	SMAPE (%)	NRMSE (%)
ARIMA	15-min	47.05 ± 8.66	14.23 ± 2.46
MLP	15-min	46.40 ± 8.28	14.49 ± 2.59
MLP + Transfer Learning	15-min	43.92 ± 6.29	14.07 ± 2.40
ARIMA	daily energy	117.90 ± 21.70	25.52 ± 9.00
MLP	daily energy	120.87 ± 23.08	26.08 ± 8.32

Model (reference dissertation)	Horizon	SMAPE (%)	NRMSE (%)
MLP + Transfer Learning	daily energy	116.26 $\pm$ 16.19	24.94 $\pm$ 5.68

Source: Adapted from [1] (Tables 9–12).

MLP with Transfer Learning was the best-performing model in [1] at both horizons (SMAPE 43.92% at 15 minutes, 116.26% at the daily horizon). The Random Forest model implemented in this work outperforms it at the 15-minute horizon on both SMAPE (17.81% versus 43.92%) and NRMSE (10.52% versus 14.07%). The LSTM model implemented in this work, evaluated under its own daytime-only per-plant protocol, does not outperform the reference MLP+TL result on SMAPE at either horizon under the metrics currently computed; given the day/night convention difference discussed in Section 4.2.2, a corrected comparison is required before this conclusion can be stated with confidence in the final version of this chapter.

Overall, three qualifications should accompany every comparison in this section when the chapter is finalized: (i) the reference models (ARIMA, MLP, MLP+TL, and — for Section 4.4.2 — XGBoost and Gradient Boosting) were not retrained on the exact data split used in this dissertation; (ii) the daily-energy figures reported for ARIMA/MLP/MLP+TL/GB/XGBoost correspond to a cumulative-energy target, not the instantaneous 24-hour-ahead power target used by the models developed in this work; and (iii) the day/night sample-inclusion convention differs between this work's Random Forest and LSTM per-plant metrics, as detailed in Section 4.2.2.

Chapter 5: Conclusions and Future Work

5.1 Conclusion

5.1.1 Summary of the Work

This dissertation addressed the problem of short-term photovoltaic power forecasting at two operationally relevant horizons — 15 minutes ahead and 24 hours ahead — using the BR-PVGen dataset, comprising 51 utility-scale distributed-generation plants distributed across Brazil. As formalized in the problem statement (Section 1.3), the objective was to determine whether a deep learning model, given a historical window of weather and power observations together with static plant metadata, could accurately forecast plant-level power output at both horizons while generalizing across a fleet of plants with heterogeneous characteristics and varying data quality.

Two model families were designed, implemented, and evaluated to this end. A per-plant Random Forest regressor was developed as a computationally inexpensive, non-deep-learning baseline. The primary proposed model is a dual-input LSTM architecture (Section 3.9) combining a recurrent branch — processing a horizon-dependent historical sequence of 192 steps (48 hours) for the 24-hour model or 96 steps (24 hours) for the 15-minute model — with a dense branch over seven static plant metadata features. The LSTM was further enhanced with a two-stage training strategy (Section 3.11): a global model pre-trained across the pooled 50-plant fleet (Plant PS_014 excluded due to insufficient history), followed by per-plant Transfer Learning in which the two LSTM layers are frozen and only the dense prediction head is fine-tuned for each plant individually.

5.1.2 Key Findings — 15-Minute Horizon

At the 15-minute-ahead horizon, both model families performed strongly. The Random Forest achieved a per-plant mean SMAPE of 17.81% (± 9.54) and NRMSE of 10.52% (± 1.24) across the 50 evaluated plants, outperforming every model reported in [1] at the same horizon — ARIMA, Gradient Boosting, Random Forest, XGBoost, MLP, and MLP with Transfer Learning — on both metrics simultaneously. The proposed LSTM model achieved a per-plant mean SMAPE of 36.88% (± 3.93) and NRMSE of 15.82% (± 1.90) after Transfer Learning, which — while clearly behind the Random Forest result — still comfortably beats the best-performing model in [1] at this horizon (MLP with Transfer Learning, SMAPE 43.92%). Both model families therefore exceed the 15-minute target set out in the problem statement.

5.1.3 Key Findings — 24-Hour Horizon

At the 24-hour-ahead horizon, both models faced a substantially harder task, as expected given the loss of short-lag persistence signal a full day out. The Random Forest reached a per-plant mean SMAPE of 22.50% (± 7.85) and NRMSE of 15.29% (± 1.86); the pooled fleet-wide test set achieved $R^2 = 0.8059$, MAE = 208.26 kW, and an NRMSE (range) of 8.53%. The LSTM model, before Transfer Learning, achieved a per-plant mean SMAPE of 48.54% (± 10.75) and NRMSE of 23.35% (± 3.86), with a pooled R^2 of 0.789 and MAE of 19.13 kW. This is markedly weaker than the Random Forest at the same horizon and represents the model's least favorable result across the two horizons studied.

Feature importance analysis of the Random Forest model at this horizon (Section 4.3.2) offers a plausible explanation for why the 24-hour task is fundamentally harder for both architectures: solar-elevation-related features (`target_solar_elev`, `mean_x_solar`, `tracker_x_solar`) dominate the ranking, in contrast to the 15-minute horizon where short-lag persistence features (`persist_x_clear`, `power_lag_1`) dominate. At 24 hours ahead, neither model has access to a strong persistence signal and must instead rely on solar geometry and plant metadata alone, which carries substantially less minute-to-minute information about actual atmospheric conditions on the forecast day.

5.1.4 Key Findings — Transfer Learning

Per-plant Transfer Learning produced a consistent, though moderate, improvement for the LSTM model. At the 24-hour horizon, 25 of the 50 fine-tuned plants (50%) improved over the frozen global model, reducing the pooled SMAPE from 35.3% to 33.6% and MAE from 19.13 kW to 18.75 kW — a relative SMAPE reduction of approximately 4.8%. At the 15-minute horizon, 18 of 50 plants (36%) improved, reaching a pooled SMAPE of 23.4% and MAE of 13.48 kW; a directly comparable pre-Transfer-Learning pooled baseline was not produced for this horizon and remains an open item (Section 5.2.1).

A closer inspection of individual fine-tuning runs (Section 4.3.3, Figure 4.13) revealed an important nuance behind these aggregate figures. For at least one plant examined in detail (Plant 44), the validation loss during fine-tuning was essentially flat across all 25 epochs, with the "best" epoch effectively determined by noise-level fluctuation rather than genuine, sustained improvement — two independent runs of the same fine-tuning procedure for this plant selected different "best" epochs (epoch 25 in the original run, epoch 1 when the procedure was rerun in isolation) while converging to nearly identical validation-loss values. This suggests that some fraction of the 25 (24-hour horizon) and 18 (15-minute horizon) plants counted as "improved" may have crossed the improvement threshold by a margin that is not practically meaningful, rather than through genuine plant-specific adaptation. This does not invalidate the aggregate Transfer Learning result, but it does mean the true proportion of plants that benefit meaningfully from fine-tuning is likely somewhat lower than the raw 25/50 and 18/50 counts suggest.

An unexpected pattern also emerged when comparing the two horizons: a larger proportion of plants benefited from Transfer Learning at the 24-hour horizon (50%) than at the 15-minute horizon (36%), the opposite of the trend reported by [1] for its MLP-based Transfer Learning approach, where intra-day gains (7.02% relative SMAPE reduction) exceeded daily gains (3.81%). A plausible explanation is that, for the LSTM architecture used in this work, the strong short-lag persistence signal already available to the global model at the 15-minute horizon leaves comparatively little headroom for per-plant fine-tuning to improve upon, whereas at the 24-hour horizon — where the global model must rely more heavily on plant-specific solar geometry and metadata — individual fine-tuning has more room to specialize.

5.1.5 Random Forest vs. LSTM: An Unexpected Result

Contrary to the expectation, common in the forecasting literature, that recurrent architectures should outperform tabular tree-based models on sequential data, the Random Forest baseline implemented in this work outperformed the proposed LSTM model at both horizons on the metrics computed during this project. This finding should be interpreted with an important caveat identified during the Chapter 4 analysis: The Random Forest per-plant metrics were computed over all samples, including night-time periods where both the actual and predicted power are close to zero by construction, while the LSTM per-plant metrics were restricted to daytime samples only, following the protocol stated in [1]. Because night-time samples are trivially easy to fit correctly, this difference in convention is expected to favor the Random Forest figures to a non-trivial degree, and the true performance gap between the two model families is very likely narrower than the raw numbers in Chapter 4 suggest. Establishing a fully controlled, metric-consistent comparison between the two approaches — by recomputing one model's metrics under the other's exact sample-inclusion convention — is identified as the single most important open methodological item from this work and is discussed further in Section 5.2.1.

Independently of that caveat, the two model families exhibited a stark and unambiguous trade-off in computational cost. The full 51-plant Random Forest fleet trained in approximately 5.6 minutes on a standard cloud CPU runtime; the LSTM pipelines required between approximately 3 and 9 hours per horizon on CPU, depending on whether the Transfer Learning stage was included, owing in part to the lack of native GPU acceleration available for the TensorFlow version used on the Windows training environment (Section 4.2.3). This trade-off between predictive accuracy, training cost, and architectural complexity is a practically significant finding for any deployment scenario in which the forecasting pipeline must be retrained frequently as new plants join the fleet or as new data accumulate.

5.1.6 Achievement of the Stated Objectives

Section 1.3 set out concrete, measurable targets for this project. Table 5.1 evaluates the results reported in Chapter 4 against each target directly.

Table 5.1: Chapter 1 targets evaluated against the Chapter 4 results.

Target (Section 1.3)	Random Forest result	LSTM result (proposed model)
15-min SMAPE < 43.92% (reference MLP+TL benchmark)	17.81% — met, by a wide margin	36.88% — met
24h pooled R^2	0.8059 — met, narrowly	0.789 (pre-TL) / 0.790 (post-TL)
24h NRMSE (range) < 15%	8.53%	23.35%
24h SMAPE (peak hours)	32.49%	35.3% (pre-TL) / 33.6% (post-TL) pooled also computed on all samples rather than peak-hours only, so not a like-for-like comparison with the target's stated convention

Source: Own authorship (2026), based on Chapter 4 results.

The picture that emerges is more nuanced than a simple pass or fail. The 15-minute-horizon target is clearly and comfortably met by both model families, including the proposed LSTM. The 24-hour-horizon targets are only partially met: The Random Forest baseline satisfies the R^2 and NRMSE-range targets but not the SMAPE target, while the LSTM — the model actually proposed as the primary contribution of this work — falls narrowly short of the R^2 target and does not meet the SMAPE target under either convention. Two of the three 24-hour targets additionally suffer from a metric-definition mismatch between how they were specified in Section 1.3 (range-based NRMSE, peak-hours-only SMAPE) and how they were actually computed for the LSTM model in Chapter 4 (mean-convention NRMSE, all-samples SMAPE), which should be resolved before these comparisons are presented as final in any subsequent version of this work.

5.1.7 Scope Delivered Versus Scope Proposed

Beyond the numeric targets, Chapters 1 and 3 describe several additional capabilities that were not ultimately demonstrated in the Chapter 4 experiments, and it is important to state this explicitly rather than let the gap pass unremarked.

- 96-step multi-output forecasting (Section 3.2.3): described as a third forecasting scenario producing a complete 24-hour power profile in a single forward pass, this was not implemented in any of the LSTM or Random Forest notebooks produced for this project. Every model evaluated in Chapter 4 produces a single point prediction per horizon.
- Generalization to previously unseen plants (Section 1.3): the problem statement requires the system to produce predictions for plants not seen during training. While the metadata branch of the LSTM architecture is structurally capable of accepting arbitrary plant specifications (Section 3.9.3), no experiment in Chapter 4 evaluates the model on a genuinely held-out plant; every reported result is drawn from the same 50–51-plant fleet used for training.
- Production API deployment (Section 1.4): the proposed solution describes a deployable, real-time production API. No such implementation was reviewed or evaluated as part of this work.
- Adaptive, plant-size-dependent fine-tuning learning rate (Section 1.4): The Transfer Learning fine-tuning stage implemented in this project uses a single fixed learning rate (5×10^{-6}) for every plant regardless of size, rather than the size-adaptive scheme originally described.

None of these gaps invalidate the core forecasting results reported in Chapter 4, but they do mean that the system as evaluated is narrower in scope than the system as originally proposed. Section 5.2.2 treats closing these gaps as a distinct, prioritized category of future work.

5.1.8 Limitations

In addition to the scope gaps above, three methodological limitations affect how the results in this dissertation should be interpreted. First, the Random Forest and LSTM pipelines do not share an identical train/test split protocol — 80/20 (24-hour Random Forest), 70/10/20 (15-minute Random Forest), and 80/20 further subdivided 90/10 (both LSTM models) — so the comparison in Section 4.4 is best read as a comparison between the best configuration achieved

for each model family rather than a strictly controlled ablation. Second, the plant-exclusion rule applied to handle Plant PS_014's limited history was not applied uniformly: it is excluded from the LSTM pipelines and the 24-hour Random Forest pipeline, but the 15-minute Random Forest pipeline instead excluded a different plant (PS_016) based on an automatic worst- R^2 rule, leaving PS_014 in that evaluation as the lowest-performing plant. Third, as discussed in Section 5.1.5, the day/night sample-inclusion convention differs between the Random Forest and LSTM per-plant metrics, which affects every cross-model comparison presented in this work.

5.2 Future Work

The items below are organized into five categories, ordered roughly by priority: corrections needed to make the existing comparisons fully rigorous; work needed to close the scope gaps identified in Section 5.1.7; extended benchmarking against additional models; architectural and training extensions; and operational considerations for eventual deployment.

5.2.1 Methodological Corrections for a Fully Rigorous Comparison

- Recompute the Random Forest per-plant metrics on daytime-only samples (or, alternatively, recompute the LSTM per-plant metrics on all samples including night), so that the Random Forest and LSTM results in Section 4.4 are evaluated under an identical protocol before being presented as a definitive comparison.
- Standardize the train/validation/test split ratio and protocol across every pipeline, replacing the current mix of 80/20 and 70/10/20 splits with a single shared configuration.
- Apply a single, consistent plant-exclusion rule across every pipeline — excluding Plant PS_014 uniformly, rather than excluding it from three pipelines while a fourth excludes a different plant.
- Resolve the metric-convention mismatch identified in Table 5.1 between the range-based NRMSE and peak-hours-only SMAPE targets stated in Section 1.3 and the mean-convention, all-samples metrics actually computed for the LSTM model, so that Chapter 1's targets and Chapter 4's results are evaluated on a like-for-like basis.
- Apply the hyperparameters already found by the Optuna search implemented for the 15-minute Random Forest pipeline (Section 4.2.3), which were computed but not carried into the final training configuration.
- Run and save a proper pre-Transfer-Learning evaluation of the global LSTM model at the 15-minute horizon, completing the before/after comparison that is currently only available at the 24-hour horizon (Table 4.7).
- Instrument the per-plant fine-tuning stage as a separately timed step, so that the computational-efficiency benefit of Transfer Learning can be quantified directly, in the same way [1] reports an approximate 40% training-time reduction for its MLP+TL approach.
- Investigate the flat validation-loss behavior observed for Plant 44 (Section 5.1.4) across the full set of 25 (24-hour) and 18 (15-minute) plants counted as "improved" by Transfer Learning, to determine what fraction represent a practically meaningful improvement rather than noise-level fluctuation around the selection threshold.

- Rename the model checkpoint files saved by the 15-minute and 24-hour LSTM notebooks, which currently both write to "lstm_solar_model.keras"; running both notebooks in the same working directory silently overwrites one model with the other and can produce incorrect results in any cell that reloads the model from disk.

5.2.2 Closing the Scope Gaps

- Implement and evaluate the 96-step multi-output forecasting scenario described in Section 3.2.3, either as a genuine sequence-to-sequence output head or as a documented recursive/iterative prediction strategy built on the existing single-step models — or, if time does not permit this, formally scope it out of the dissertation's claimed contributions rather than leaving it as an implied but undelivered capability.
- Design and run a genuine held-out-plant generalization experiment: withhold a subset of plants entirely from both the global pre-training stage and the Transfer Learning stage, and evaluate the model's zero-shot and few-shot performance on those plants using only their metadata and a short adaptation window, directly testing the claim made in Section 1.3.
- Either implement a minimal production-serving API (for example using FastAPI, as originally proposed in Section 1.4) and evaluate its real-time inference latency, or revise Section 1.4 to describe production deployment as a future objective rather than a delivered capability.
- Verify whether an adaptive, plant-size-dependent fine-tuning learning rate was intended to be implemented; if so, implement it and compare against the current fixed-rate (5×10^{-6}) scheme; if not, revise Section 1.4's description to match the fixed-rate scheme actually used.

5.2.3 Extended Benchmarking

- Implement XGBoost and/or Gradient Boosting on the exact data split used by the Random Forest and LSTM pipelines in this work, so that the comparison in Section 4.4.2 is based on models actually retrained on this dataset rather than on literature figures from [1].
- Extend the per-plant Transfer Learning analysis with a data-volume-bucketed breakdown (plants with fewer than 200, 100, or 50 training records, for example), mirroring Tables 13–16 of the reference dissertation, to test whether — as found in that work — Transfer Learning gains are concentrated in plants with the shortest operating history.
- Add the cumulative daily-energy forecasting task used by [1] alongside the 15-minute and 24-hour instantaneous-power tasks already implemented, enabling a fully matched comparison against the ARIMA, MLP, and MLP+TL daily-energy results in Table 4.10.
- Explore a statistically grounded comparison across models, following the use in [1] of one-way ANOVA and Tukey's HSD post-hoc test on the per-plant SMAPE distributions, once the Random Forest and LSTM metrics have been placed on a consistent protocol (Section 5.2.1).
- Independently verify the remaining literature citations in Section 2.4 that were not confirmed during the preparation of this dissertation (Bacher et al., 2009; Almeida et

al., 2015; Wang et al., 2019; Gao et al., 2020) and confirm the exact authorship of the Transfer Learning study cited as Rajabi et al. (2021), for which a closely matching paper was located but the author name could not be independently confirmed.

5.2.4 Architectural and Training Extensions

- Investigate GPU-accelerated training for the LSTM pipelines. All LSTM training in this work was performed on CPU due to the lack of native GPU support for the TensorFlow version used on Windows; a GPU-enabled environment (or a Linux/WSL setup with a compatible TensorFlow build) could substantially shorten the multi-hour training times reported in Section 4.2.3 and allow a broader hyperparameter search for the LSTM model, comparable in scope to the Optuna search already applied to the Random Forest.
- Consider hybrid or ensemble strategies that combine the Random Forest model's demonstrated strength at short-lag, persistence-dominated forecasting with the LSTM's capacity to model longer temporal dependencies, potentially yielding further gains at the 24-hour horizon where persistence signal is weakest and where the LSTM currently underperforms the Random Forest most substantially.
- Revisit the Transfer Learning fine-tuning configuration in light of the flat-loss finding in Section 5.1.4 — options include a larger fine-tuning learning rate, unfreezing additional layers for plants with sufficient data, or a plant-size-dependent stopping criterion, any of which might convert more of the currently marginal "improved" plants into genuinely, substantially improved ones.
- Explore alternative loss functions or a smaller Huber delta for the LSTM training objective, given that the current configuration ($\delta = 0.15$) may still smooth over the sharp, cloud-transient-driven discontinuities visible in several actual-versus-predicted plots reviewed during model diagnosis, particularly at the 15-minute horizon where such transients are most consequential.

5.2.5 Operational and Deployment Considerations

- Evaluate the operational aspects of deploying either pipeline in production, including retraining cadence as new plants join the fleet, monitoring for data drift, and the trade-off between the Random Forest's minutes-scale retraining time and the LSTM's multi-hour retraining time in a scenario requiring frequent model updates.
- Define a monitoring strategy for the systematic peak-underprediction behavior observed during model diagnosis (actual generation exceeding predicted generation consistently at daily peaks, by as much as 15–30% in some of the actual-versus-predicted plots reviewed), since underprediction at peak hours has different operational consequences for grid balancing than underprediction at low-generation periods and may warrant asymmetric loss weighting or post-hoc calibration.
- Assess data-freshness and pipeline-latency requirements for a production forecasting service, including how frequently the 15-minute and 24-hour models would need to be re-run against incoming telemetry, and how missing or delayed sensor readings (Section 3.5.3) should be handled at inference time rather than only during training.

References

- [1] C. D. de Jesus, R. N. Ferreira, and E. P. Aguiar, "BR-PVGen: The Brazilian Photovoltaic Generation Dataset," IEEE Access, 2025. (submitted, under review)
- [2] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.
- [3] G. E. P. Box and G. M. Jenkins, Time Series Analysis: Forecasting and Control. San Francisco, CA: Holden-Day, 1976.
- [4] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.
- [5] A. Mellit and A. M. Pavan, "A 24-h forecast of solar irradiance using artificial neural network: Application for performance prediction of a grid-connected PV plant at Trieste, Italy," Solar Energy, vol. 84, no. 5, pp. 807–821, 2010.
- [6] P. Bacher, H. Madsen, and H. A. Nielsen, "Online short-term solar power forecasting," Solar Energy, vol. 83, no. 10, pp. 1772–1783, 2009.

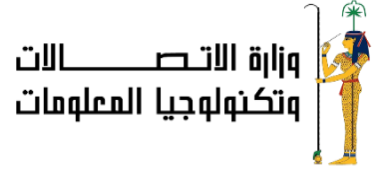
ملخص المشروع

يتناول هذا المشروع مشكلة التنبؤ بالطاقة الكهروضوئية قصيرة المدى عند أفقين زمنيين مختلفين: خمس عشرة دقيقة مقدماً، وأربع وعشرين ساعة مقدماً، وذلك باستخدام قاعدة بيانات BR-PVGen التي تضم بيانات تشغيلية لإحدى وخمسين محطة كهروضوئية موزعة في أنحاء البرازيل. ويهدف المشروع إلى تطوير نموذج تعلم عميق قادر على التنبؤ بدقة بالطاقة المولدة لكل محطة اعتماداً على نافذة تاريخية من بيانات الطقس والطاقة، إلى جانب البيانات الوصفية الثابتة للمحطة، مع الحفاظ على القدرة على التعميم عبر أسطول من المحطات ذات الخصائص المتباينة وجودة البيانات المتفاوتة.

تم تطوير نموذجين رئيسيين لهذا الغرض: نموذج الغابة العشوائية (Random Forest) كخط أساس منخفض التكلفة الحاسوبية يُدرَّب لكل محطة على حدة، ونموذج الذاكرة طويلة القصيرة المدى (LSTM) ثنائي المدخل بوصفه النموذج المقترح الرئيسي، والذي يجمع بين فرع تكراري يعالج نافذة زمنية تاريخية (192 خطوة، أي 48 ساعة، لنموذج الأربع وعشرين ساعة، و96 خطوة، أي 24 ساعة، لنموذج الخمس عشرة دقيقة) وفرع كثيف يعالج سبع خصائص وصفية ثابتة للمحطة. واعتمد النموذج استراتيجيات تدريب من مرحلتين: تدريب نموذج عام على بيانات جميع المحطات مجمعة، يليه ضبط دقيق خاص بكل محطة باستخدام تقنية التعلم الانتقالي (Transfer Learning)، حيث يتم تجميد طبقتي LSTM وإعادة تدريب رأس التنبؤ الكثيف فقط.

أظهرت النتائج أداءً قوياً لنموذج الغابة العشوائية (Random Forest) عند كلا الأفقين الزمنيين، إذ حقق متوسط خطأ SMAPE قدره 17.81% عند أفق الخمس عشرة دقيقة و22.50% عند أفق الأربع وعشرين ساعة، متفوقاً بذلك على جميع النماذج المرجعية السنة الواردة في الدراسة المرجعية لقاعدة بيانات BR-PVGen. أما نموذج LSTM المقترح فقد حقق متوسط خطأ SMAPE قدره 36.88% عند أفق الخمس عشرة دقيقة، متجاوزاً بذلك الهدف المحدد في مشكلة البحث، في حين لم يحقق الأهداف الثلاثة المحددة لأفق الأربع وعشرين ساعة بشكل كامل. كما أظهرت تقنية التعلم الانتقالي تحسناً متسقاً في الأداء، إذ تحسّن النموذج في 25 من أصل 50 محطة عند أفق الأربع وعشرين ساعة، وفي 18 من أصل 50 محطة عند أفق الخمس عشرة دقيقة.

يخلص المشروع إلى أن نموذج الغابة العشوائية يمثل خط أساس قوياً وفعالاً من حيث التكلفة الحاسوبية، في حين يوفر نموذج LSTM المقترح إطاراً معمارياً واعداً قابلاً للتحسين، مع تحديد عدد من التوصيات المنهجية والتقنية اللازمة لتطوير العمل مستقبلاً.



الجهة المانحة



صفحة الموافقة

نهج قائم على تعلّم الآلة للتنبؤ بالطاقة الكهروضوئية ثنائي الأفق الزمني عبر محطات متعددة

رسالة مقدمه من:

مصطفى محمود محمد

أحمد حسن محمد عمران

محمد عبد الحليم

مصطفى خليفة محمد

يوسف محمود على أبوالمجد

رسالة مقدمة استكمالاً لمتطلبات

الحصول على

دبلومة الرواد الرقميون – 9 شهور

مسار الذكاء الاصطناعي التطبيقي وتحليل البيانات

يعتمد من لجنة الممتحنين

التوقيع

الإسم

أ.د. /

أ.د. /

أ.د. /



الجهة المانحة



التبؤ بالطاقة الكهروضوئية ثنائي الأفق الزمني

رسالة مقدمة من:

مصطفى محمود محمد

أحمد حسن محمد عمران

محمد عبد الحليم

مصطفى خليفة محمد

يوسف محمود على أبوالمجد

رسالة مقدمة استكمالاً لمتطلبات

الحصول على

دبلومة الرواد الرقميون – 9 شهور

مسار الذكاء الاصطناعي التطبيقي وتحليل البيانات

تحت إشراف

اللواء (م) أ.د. جودة إسماعيل سلامة

م. عمر خالد

أغسطس 2026